

Survey paper

Vision transformers on the edge: A comprehensive survey of model compression and acceleration strategies

Shaibal Saha^{*,} Lanyu Xu^{id}

Department of Computer Science and Engineering, Oakland University, Rochester Hills, 48309, MI, USA

ARTICLE INFO

Communicated by W. Zhou

Keywords:

Vision transformer
Model compression
Pruning
Quantization
Hardware accelerators
Edge computing

ABSTRACT

In recent years, vision transformers (ViTs) have emerged as powerful and promising techniques for computer vision tasks such as image classification, object detection, and segmentation. Unlike convolutional neural networks (CNNs), which rely on hierarchical feature extraction, ViTs treat images as sequences of patches and leverage self-attention mechanisms. However, their high computational complexity and memory demands pose significant challenges for deployment on resource-constrained edge devices. To address these limitations, extensive research has focused on model compression techniques and hardware-aware acceleration strategies. Nonetheless, a comprehensive review that systematically categorizes these techniques and their trade-offs in accuracy, efficiency, and hardware adaptability for edge deployment remains lacking. This survey bridges this gap by providing a structured analysis of model compression techniques, software tools for inference on edge, and hardware acceleration strategies for ViTs. We discuss their impact on accuracy, efficiency, and hardware adaptability, highlighting key challenges and emerging research directions to advance ViT deployment on edge platforms, including graphics processing units (GPUs), application-specific integrated circuit (ASICs), and field-programmable gate arrays (FPGAs). The goal is to inspire further research with a contemporary guide on optimizing ViTs for efficient deployment on edge devices.

1. Introduction

Deep learning architectures have evolved significantly in recent years, with transformers emerging as one of the most transformative breakthroughs. Transformers initially introduced for natural language processing (NLP) by Vaswani et al. [1] in 2017 replaced recurrent models such as long short-term memory (LSTMs) [2] and gated recurrent [3], leveraging self-attention mechanisms to capture long-range dependencies in sequential data efficiently.

Following the tremendous success of transformers in NLP, researchers adapted their architecture for computer vision (CV), leading to the development of vision transformers (ViTs) [4]. Unlike convolutional neural networks (CNNs) [5], which rely on hierarchical feature extraction, ViT model processes images as sequences of patch embeddings, enabling global context modeling via self-attention. Since the introduction of ViTs [4], research interest in ViT-based models has grown exponentially, as reflected in the increasing number of publications each year (see Fig. 1a). This surge in publications highlights ViTs' dominance in CV tasks, driven by their state-of-the-art (SOTA) performance across various tasks, including image classification [4,6], object detection [7,8], and segmentation [9,10].

While ViT-based models have demonstrated significant capabilities, the substantial size of these models presents major challenges for practical deployment. For instance, ViT-Huge includes over 632M parameters [4] and has recently been extended to 22B parameters [11], demanding extensive computational resources. These memory and processing requirements make direct deployment on resource-constrained edge devices impractical without optimization. To overcome these limitations, researchers have explored various model compression techniques to reduce computational overhead while preserving performance. As ViTs continue to gain prominence in CV tasks (Fig. 1a), there has been a parallel increase in research focused on optimizing their efficiency through compression techniques (Fig. 1b). Techniques such as pruning [12,13], quantization [14,15], and knowledge distillation (KD) [16,17] on ViT have gained traction, offering solutions to reduce model size, improve inference speed, and lower power consumption without significantly compromising accuracy. However, compression techniques alone are often insufficient to meet real-time applications' stringent latency and throughput requirements on edge devices.

Although model compression techniques reduce computational overhead, they are often insufficient to ensure real-time performance on

* Corresponding author.

E-mail address: shaibalsaha@oakland.edu (S. Saha).

<https://doi.org/10.1016/j.neucom.2025.130417>

Received 26 February 2025; Received in revised form 16 April 2025; Accepted 30 April 2025

Available online 16 May 2025

0925-2312/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

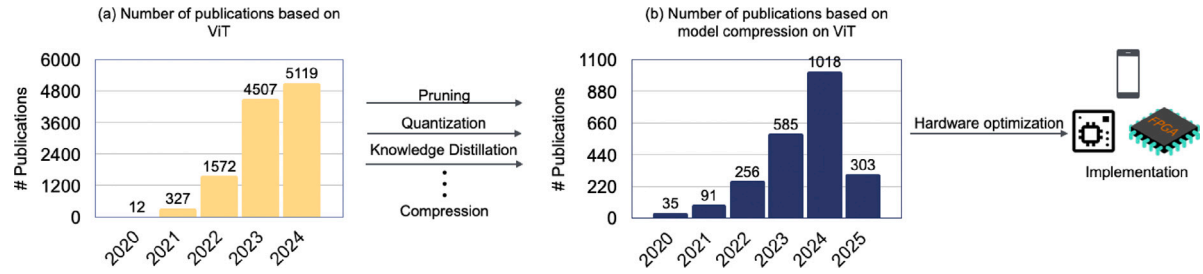


Fig. 1. (a) The prevalence of transformer-based models in computer vision has led to a substantial increase in research publications. (b) Given their high computational complexity, model compression techniques are critical for reducing redundancy and improving efficiency. These advancements are essential for optimizing ViTs for hardware acceleration and real-world deployment on resource-constrained platforms [18].

edge devices due to hardware-specific constraints. Effective deployment of ViT requires tailored acceleration strategies that align with the architectural characteristics of each hardware platform. These strategies optimize execution by addressing ViTs' core computational bottlenecks – such as self-attention's quadratic complexity and patch embedding inefficiencies – through hardware-aware optimizations, efficient non-linear operations, and resource-efficient computation. These approaches address the inherent on-device computational bottlenecks of ViTs, such as the quadratic complexity of self-attention and the inefficiencies in processing patch embeddings. Recent advancements in accelerating techniques, including the use of specialized or custom accelerators and optimized libraries (e.g., TensorRT), have further expanded the possibilities for accelerating ViTs on edge devices (e.g., graphics processing units (GPUs), application-specific integrated circuits (ASICs), and field-programmable gate arrays (FPGAs)). By bridging the gap between model-level optimizations and hardware-specific execution, software-hardware (SW-HW) co-design also plays a pivotal role in deploying ViTs on edge devices [19–21]. Compression techniques, optimized software tools, and hardware-aware acceleration strategies [21–23] provide a pathway toward efficient, low-latency ViT inference, unlocking new possibilities for different application domains such as autonomous driving, mobile vision applications, and medical imaging on edge devices.

This survey provides a comprehensive review of both model compression and acceleration strategies tailored for ViT, focusing on their applicability to edge devices such as GPUs, central processing units (CPUs), FPGAs, and ASICs. We systematically categorize and analyze the latest advancements in pruning, quantization, KD, and hardware-aware optimizations. Furthermore, we explore emerging acceleration techniques, which aim to reduce latency and improve energy efficiency. By synthesizing insights from a broad range of studies, this survey serves as a valuable resource for researchers and practitioners seeking to deploy ViTs on edge devices.

1.1. Motivations and contribution

ViTs have revolutionized CV tasks, achieving SOTA performance across tasks such as image classification, object detection, and segmentation. However, their high computational cost, memory footprint, and energy consumption present significant challenges for deployment on resource-constrained edge devices. While various optimization techniques exist for ViTs, a systematic survey that collectively examines ViT-focused model compression, software tools, evaluation metrics, and hardware acceleration strategies for edge deployment remains limited. Existing surveys often treat these aspects independently, lacking a single resource that presents them together in a structured manner tailored to edge deployment. Table 1 compares existing surveys on ViT model compression and acceleration techniques.

This survey addresses this gap by systematically analyzing model compression techniques (pruning, quantization, KD) and hardware-aware acceleration strategies (efficient attention mechanisms, SW-HW co-design, FPGA optimizations, etc.). By analyzing insights from a

broad range of studies, this survey serves as a valuable resource for researchers seeking to deploy ViTs on edge devices. The main contributions of our survey are as follows:

1. We systematically categorize and analyze model compression techniques such as pruning, quantization, and KD to optimize ViTs for deploying on edge devices.
2. We investigate current software tools for efficient inference and hardware-aware accelerating techniques to enhance inference efficiency across edge platforms like GPUs, FPGAs, and ASICs for ViT models.
3. By providing a structured roadmap for integrating compression and acceleration techniques, we offer comparative analyses and identify challenges and future research directions for real-time, low-power ViT applications.

1.2. Literature collection and organizations

Our literature search was conducted across major academic databases, including **Google Scholar**, **IEEE Xplore**, **arXiv**, and the **ACM Digital Library**, to ensure comprehensive coverage of relevant research. We utilized targeted search queries with keywords such as **vision transformer**, **acceleration techniques**, **edge devices**, **software-hardware co-design**, **pruning**, **quantization**, and **knowledge distillation** to identify studies relevant to this survey. A total of 170 papers were collected, with works published up to January 2025 considered for inclusion. Fig. 2 presents a structured overview of the selected literature, categorizing each referenced work according to the core research themes and enabling tools addressed in this survey. Papers were included based on how well they aligned with our key research questions. A paper was selected if it met one or more of the following criteria:

1. Does the paper propose a compression technique for improving ViT efficiency in terms of computational cost or energy consumption?
2. Does the paper provide a comparative analysis of ViT acceleration techniques or benchmark performance across different hardware platforms?
3. Does the paper explore the integration of ViTs with hardware-aware optimizations, including SW-HW co-design strategies?

The remainder of the survey is organized as follows. Section 2 presents an in-depth discussion on model compression techniques, including pruning, quantization, and KD, which enhance ViT efficiency while preserving performance across various CV tasks. Following this, Section 3 explores an overview of current software tools, optimization frameworks, and evaluation metrics designed for efficient edge inference across different edge devices. Section 4 further explores hardware-aware acceleration techniques, focusing on optimizations for non-linear operations (e.g., softmax, GELU, and LayerNorm) and recent SOTA SW-HW co-design strategies, along with a comparative analysis of

Table 1

Comparison of existing surveys on model compression and acceleration techniques for ViTs. ✓✓ indicates a comprehensive discussion, while ✓ denotes a limited discussion (see [24–27]).

Survey	Year	Scope			
		Model compression	Software tools	Evaluation metrics	Hardware accelerators
[24]	2022	✓	✗	✗	✗
[25]	2022	✓✓	✗	✗	✓
[26]	2024	✓✓	✗	✓	✗
[27]	2024	✓✓	✗	✗	✓
Our survey	2025	✓✓	✓✓	✓✓	✓✓

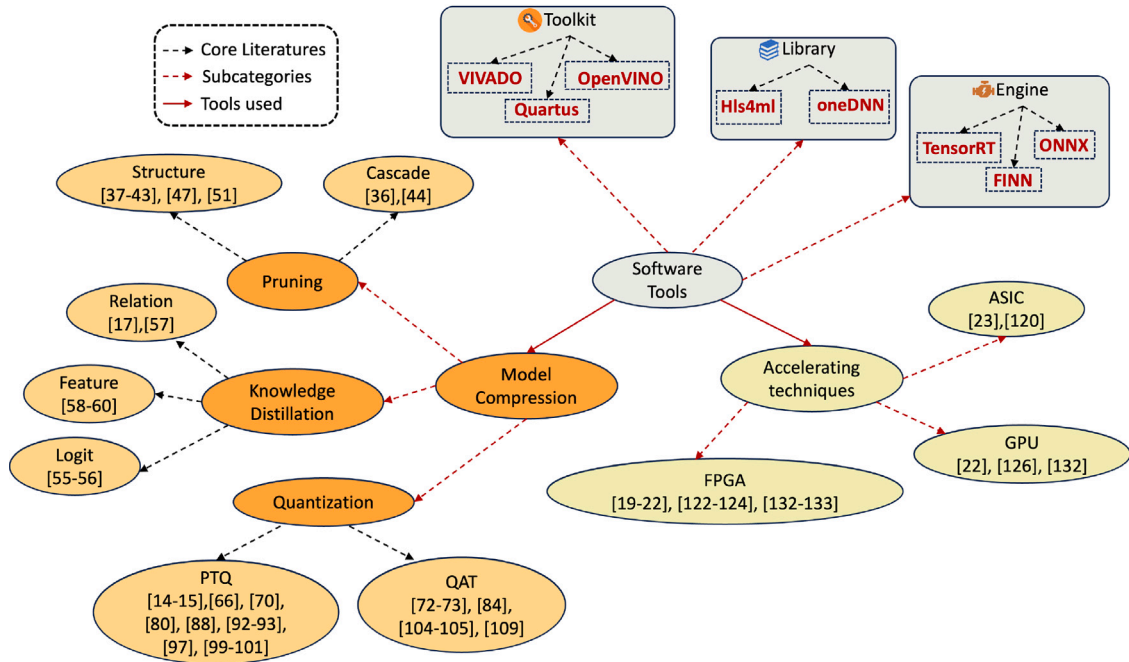


Fig. 2. Core components and tools are covered in this survey. This survey focuses on three main parts—model compression techniques (pruning, KD, and quantization), accelerating techniques (using FPGA, GPU, and ASIC), and associated software tools (toolkits, libraries, and inference engines)—to provide a comprehensive understanding of efficient ViT deployment.

their performance. Furthermore, Section 5 discusses key challenges and future research directions, identifying multiple avenues for advancing ViT acceleration and deployment on edge devices. Finally, we conclude this paper in Section 6.

2. Model compression

Model compression is a key technique for deploying a model on edge devices with limited computational power and memory while maintaining accuracy. It enables lowering inference latency as well as reducing memory and energy consumption. However, model compression on ViT still needs extensive exploration due to its complex architecture and high resource usage tendencies. In this section, we will discuss prominent compression techniques for ViT models: pruning (Section 2.1), KD (Section 2.2), and quantization (Section 2.3).

2.1. Pruning

Pruning is used to reduce both memory and bandwidth. Most of the initial pruning techniques based on biased weight decay [28], second-order derivatives [29], and channels [30]. Early days pruning techniques reduce the number of connections based on the hessian of the loss function [31,32]. In general, pruning removes redundant parameters that do not significantly contribute to the accuracy of results. The pruned model has fewer edges/connections than the original model. Most early pruning techniques are like brute force pruning,

where one needs to manually check which weights do not cause any accuracy loss. Pruning techniques in deep learning became prominent post-2000 as neural networks (NNs) grew in size and complexity. The following subsections will discuss different pruning types and recent techniques applied to ViT-based models.

2.1.1. Types

In recent studies, various pruning techniques have been utilized to optimize ViT models, categorizing them into different methods based on their approach and application timing. For example, unstructured pruning targets individual weights for removal, whereas structured pruning removes components at a broader scale, like layers or channels. On the one hand, static pruning is predetermined and fixed, which is ideal for environments with known constraints. On the other hand, dynamic pruning offers real-time adaptability, potentially enhancing model efficiency without sacrificing accuracy. Another pruning method, cascade pruning, is highlighted as a hybrid approach, integrating the iterative adaptability of dynamic pruning with the structured approach of static pruning. The following subsections will be a detailed discussion of different pruning types.

Unstructured vs. Structured Pruning Unstructured pruning removes individual weights or parameters from the network based on certain criteria. Unstructured pruning can result in highly sparse networks. However, it often does not lead to computational efficiency as the sparsity is not aligned with the memory access patterns or computational

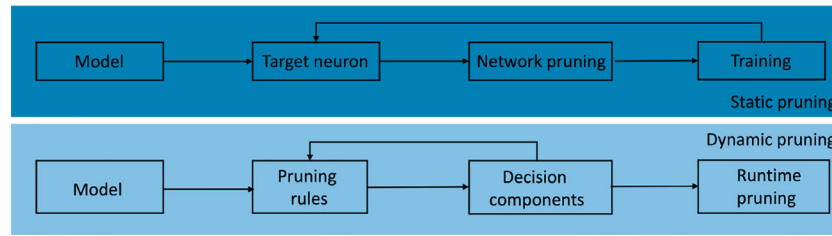


Fig. 3. Static pruning vs. Dynamic pruning.

primitives of hardware accelerators. However, structured pruning applies to the specific components of the network, especially in layers, neurons, or channels. Structured pruning leads to more hardware-friendly sparsity patterns but often at the cost of higher accuracy loss. Cai et al. [33] proposed a two-stage coarse-grained/fine-grained structured pruning method based on top-K sparsification, reducing 60% overall computation in the embedded NNs. In a recent survey [34], He et al. discussed a range of SOTA structured pruning techniques, covering topics such as filter ranking methods, regularization methods, dynamic execution, neural architecture search (NAS), the lottery ticket hypothesis, and the applications of pruning.

Static vs. Dynamic Pruning Fig. 3 demonstrates the workflow of the static and dynamic pruning. Static pruning works at the offline inference level, whereas dynamic pruning performs at the runtime level. Moreover, static pruning applies during training, where a fixed portion of a NN's components, such as neurons, channels, or weights, is removed or pruned. In static pruning, the decision on which components to prune is typically made before the training begins, and the pruning schedule remains constant throughout training. Static pruning is helpful in scenarios where the hardware constraints are well-defined. One of the most used static pruning techniques is Magnitude-based pruning [35]. In magnitude-based pruning, given a pruning rate r , weights whose absolute value is among the smallest $r\%$ are pruned. In other words, for a weight matrix W of a layer, weights w in W are pruned if $|w| \leq \text{threshold}$, where the **threshold** is determined such that the proportion of $|w| \leq \text{threshold}$ is r . In contrast, dynamic pruning applies during runtime based on specific criteria, such as the importance of neurons or weights. One of the significant drawbacks of static pruning is that it relies on a fixed pruning schedule and rate, which is determined before training begins. That means that static pruning does not adjust to the network's learning progress or the changing importance of neurons or weights during training. Dynamic pruning evaluates and adjusts the pruning criteria during training based on real-time importance assessments to overcome the limitation of static pruning. Most of the recent pruning techniques use dynamic pruning techniques [13,36,37] to get the accuracy without loss of any information.

Cascade Pruning Cascade pruning combines the iterative nature of dynamic pruning with predefined aspects resembling static pruning. Cascade pruning operates through multiple sequential iterations, also known as stages. Each stage selects a predefined portion of the network's components, such as neurons, channels, or weights, for pruning. The criteria for choosing which components to prune can vary between iterations. The ability to adjust pruning criteria between iterations makes cascade pruning adaptable to evolving training data, tasks, or hardware constraints.

2.1.2. Pruning techniques for vision transformer

Pruning methods for ViT-based models remain an underexplored area, with only a handful of studies in recent years. Current ViT pruning methods can be mainly divided into importance-based, token pruning, structure pruning, cascading pruning (combining multiple pruning techniques for each module), and miscellaneous methods. Note

that many of these pruning approaches overlap across categories. For instance, patch slimming [39] and CP-ViT [36] apply both importance-based and structured token pruning, while CP-ViT [36] and VTC-LFC [44] also follow a cascade strategy. Table 2 summarizes the categorization across different pruning approaches. This section provides a brief overview of the current SOTA pruning techniques for ViTs.

Structure Pruning Recently, many studies on ViT pruning have embraced structure pruning techniques to optimize model efficiency. As discussed in the previous subsections, importance-based pruning and token pruning can also be considered as structured pruning. Additionally, Yu et al. [41] proposed a structure pruning in a ViT named UVC, where they pruned the number of heads and dimensions inside each layer. Experiments of this paper were conducted in various ViT models (e.g., DeiT-Tiny and T2T-ViT-14) on ImageNet-1k [45] datasets. DeiT-Tiny [46] cut down to 50% of the original floating-point operations per second (FLOPS) while not dropping accuracy much in this study. Another study [47] proposed structure pruning on multi-head self-attention (MSA) [1] and feedforward neural network (FFN) [1] by removing unnecessary parameter groups. Other studies on structure pruning named NViT [42] proposed hessian-based iterative structure pruning criteria comparable across all layers and structures. Moreover, it incorporated latency-aware regularization techniques to reduce latency directly. Another study on the structure pruning in ViT called S²ViT [43] removed submodules like self-attention heads by manipulating the weight, activations & gradients. Additionally, current structure pruning methods on ViTs utilize either importance-based or learnable token selector pruning.

Importance-based Pruning One of the significant challenges in importance-based pruning for ViTs lies in determining effective thresholds to identify which components – such as channels, attention heads, blocks, or input patches – should be pruned. Different pruning strategies address this by designing tailored scoring criteria and thresholding mechanisms, often customized based on the targeted layer type or pruning granularity. Since a significant portion of ViT's computational burden stems from the MSA and multi-layer perceptron (MLP) modules [1], targeted pruning of these modules can yield substantial computational savings. Fig. 4 summarizes the current pruning techniques on ViT based on different important score measurements. Zhu et al. [38] introduced vision transformer pruning (VTP), one of the first dedicated pruning methods for ViTs that removes dimensions with lower importance scores, achieving a high pruning ratio without sacrificing accuracy. VTP operates in three key steps: (1) L1 sparse regularization is applied during training to identify less significant channels, (2) channel pruning eliminates redundant computations based on thresholds derived from a predefined pruning rate, and (3) finetuning. However, VTP ignores network depth, which is an important dimension for pruning the model. To address this issue, another study [40] (see Fig. 4(a–c)) explored width and depth pruning on the MSA module in the transformer models. Block a in Fig. 4 illustrates the application of binary masking N within the MSA module. Additionally, Blocks b and c in Fig. 4 depict width pruning applied to the linear projection matrices and attention head pruning, respectively. Moreover, leveraging the learnable saliency score approach enables non-uniform sparsity across layers

Table 2

Categorization of ViT pruning methods across key perspectives. Many methods span multiple categories.

Method	Importance-Based	Token pruning	Structured	Cascade
VTP [38]	✓	✗	✗	✗
Patch Slimming [39]	✓	✓	✗	Top-down
WDPruning [40]	✓	✗	✓	✗
SP-ViT [37]	✗	✓	✓	✗
UVC [41]	✓	✗	✓	✗
CP-ViT [36]	✓	✓	✓	✓
NViT [42]	✓	✗	✓	✗
S ² ViT [43]	✓	✓	✓	✗
VTC-LFC [44]	✓	✗	✓	✓

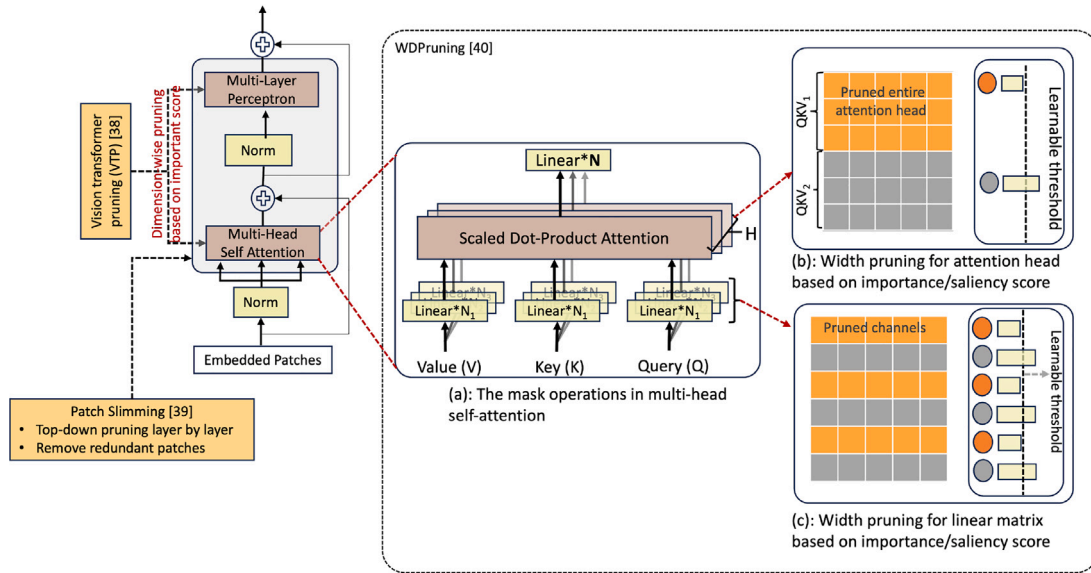


Fig. 4. The overview of importance-based pruning techniques for ViTs. VTP [38] prunes dimensionality in the multi-head self-attention (MSA) and multi-layer perceptron (MLP) modules using important scores. WDPruning [40] applies a binary mask (N) to the MSA module (a), followed by pruning of (b) attention heads and (c) linear projection channels. Patch Slimming [39] removes redundant patches in a top-down, layer-wise manner.

and facilitates progressive block removal, allowing the model to be pruned sequentially during a single training epoch. Furthermore, Tang et al. [39] identified that many input patches exhibit redundancy across transformer layers. Based on this insight, they proposed a layer-by-layer patch-slimming approach that reduces redundant patches in a top-down manner, offering a more adaptive alternative to conventional structured pruning techniques. The authors calculated their importance scores for the final classification feature to identify unimportant patches. The proposed method started by identifying important patches in the last transformer layer and recursively utilized them back to patches in earlier layers.

Token Pruning Kong et al. [37] introduced a latency-aware soft token pruning framework, SP-ViT. Fig. 5 illustrates the workflow of SP-ViT token pruning on ViT models. The framework was implemented on vanilla transformers such as data-efficient image transformers (DeiT) [46] and swin transformers [6]. The authors proposed a dynamic attention-based multi-head token selector for adaptive instance-wise token selection. Subsequently, they incorporated a soft pruning method that consolidated less informative tokens into a package token instead of entirely discarding them, identified by the selector module. The authors evaluated their proposed method on ImageNet-1k [45] dataset using Swin-S, Swin-S/T [6], PiT-Xs/S [48] models. SP-ViT was subsequently adopted by FPGA-based accelerating frameworks such as HeatViT [20]. While SP-ViT [37] was initially developed and evaluated for image classification, recent studies have extended token pruning techniques to more complex tasks such as object detection [49] and semantic segmentation [50] using lightweight, background-aware ViTs.

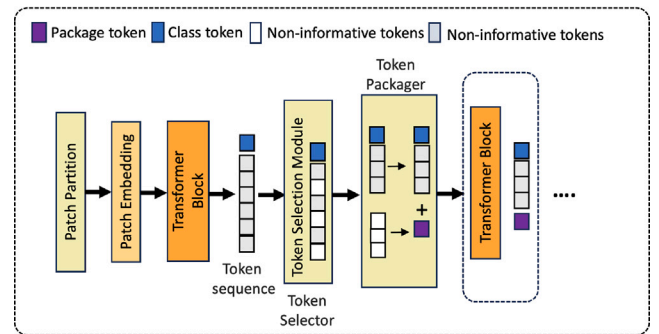


Fig. 5. The overview of token pruning based on SP-ViT [37].

Cascade Pruning Cascade pruning combines multiple pruning techniques to reduce parameters and giga FLOPS (GFLOPS) while preserving accuracy. A standout method named VTC-LFC [44] aimed to improve the identification of informative channels and tokens in a model, leading to better accuracy preservation. This approach introduced a bottom-up cascade (BCP) pruning strategy that gradually prunes tokens and channels, starting from the first block and advancing to the last. The pruning process is controlled by a hyperparameter called a **global allowable drop**, ensuring the performance drop remains within an

Table 3

Top-1 (%) accuracy comparison of pruning techniques across different ViT models. ‘↓’ denotes accuracy loss from the baseline ViT model, and ‘↑’ denotes the accuracy gain after applying the pruning technique. All results are evaluated on the ImageNet-1k [45] dataset.

Algorithm	Method	Top-1 accuracy (%) on ViT models				
		DeiT-T	DeiT-S	DeiT-B	Swin-S	T2T-ViT-14
Baseline	–	72.2	79.8	81.8	83.2	81.5
VTP [38]	Dimension-based	–	–	80.7 (↓ 1.34%)	–	–
WDPruning [40]	Width-depth pruning	70.34 (↓ 2.58%)	78.3 (↓ 1.88%)	80.8 (↓ 1.22%)	81.8 (↓ 1.68%)	–
MD [51]	Dependency based pruning by Gaussian-search	–	79.9 (↑ 0.13%)	82.3 (↓ 0.61%)	–	81.7 (↑ 0.24%)
SPViT [37]	Pruning based on token score	–	–	–	82.7 (↓ 0.60%)	–
PS-ViT [39]	Layer-wise top down pruning	72.0 (↓ 0.28%)	79.4 (↓ 0.50%)	81.5 (↓ 0.30%)	–	81.1 (↓ 0.49%)
UVC [41]	Head/dimension pruning	70.6 (↓ 2.08%)	78.8 (↓ 1.25%)	80.6 (↓ 1.47%)	–	78.9 (↓ 3.19%)
NViT [42]	Latency-aware, Hessian- based pruning	76.2 (↑ 5.55%)	82.2 (↑ 3.01%)	83.3 (↑ 1.83%)	83.0 (↓ 0.24%)	–
SAViT [47]	Structural pruning on MSA attention & FFN	70.7 (↓ 2.08%)	80.1 (↑ 0.37%)	81.7 (↑ 0.12%)	–	–
S ² ViTE [43]	Removing sub-modules by manipulating weight, activation,& gradient	70.1 (↓ 2.91%)	79.2 (↓ 0.75%)	82.2 (↑ 0.49%)	–	–
VTC-LFC [44]	Token & channel pruning using a hyperparameter	71.0 (↓ 1.66%)	79.6 (↓ 0.25%)	81.6 (↓ 0.24%)	–	–
CP-ViT [36]	Prune PH-regions in MSA & FFN dynamically	71.2 (↓ 1.39%)	79.0 (↓ 1.00%)	81.1 (↓ 0.86%)	–	–

PH-region: Informative Patches and attention Heads identified during cascade pruning.

acceptable range. Additionally, BCP guarantees efficient compression without sacrificing model performance by pruning each block and immediately stopping the compression process when the performance drop reaches a predefined threshold. Another cascade ViT pruning [36] utilized the sparsity for pruning PH-regions (containing patches and heads) in the MSA & FFN progressively and dynamically. The authors conducted experiments on three different types of datasets: ImageNet-1k [45], CIFAR-10 [52], and CIFAR-100 [52].

Miscellaneous Approaches in Pruning Hou et al. [51] introduced a multi-dimensional pruning strategy for ViTs, leveraging a statistical dependence-based criterion to identify and remove redundant components across different dimensions. Beyond this, several pruning techniques have been developed to accelerate ViTs, particularly for edge devices, including column balanced block pruning [53], end-to-end exploration [43], gradient-based learned runtime pruning [54]. These techniques have shown stability in applying pruning on ViT models without compromising accuracy.

2.1.3. Discussion

Pruning is utilized as a fundamental way to reduce the computation of the pre-trained ViT models. For ViT, the development of the pruning methods has systematically covered each perspective of model design, making the current pruning methods more flexible and well-organized for ViT models. We provide a comparative summary of these pruning methods in terms of Top-1 accuracy (Table 3), GFLOPS reduction (Table 4), and parameter reduction in millions (Table 5), comparable to baseline models. We inherit the baseline results from the official implementations of the respective models, as detailed in the tables, and compute the corresponding accuracy drop, parameter reduction, and GFLOPS reduction rates to evaluate the impact of pruning. Most pruning methods evaluated in Tables 3–5 focus on the ImageNet-1K dataset [45], with CP-ViT [36] being a notable exception, tested on CIFAR [52] dataset. As shown in Table 3, NViT [42] consistently achieves competitive top-1 (%) accuracy across DeiT-Tiny (T), Small

(S), Base (B), and Swin-S, with a minimal drop of only 5.55% from the baseline. Interestingly, few studies (e.g., SPViT, VTP) have explored pruning across diverse ViT models such as Swin or T2T-ViT-14, highlighting a gap in the generalizability of current techniques. In terms of computational savings, UVC [41] achieves the highest GFLOPS reduction for DeiT-T/S variants. Notably, NViT also delivers substantial reductions in GFLOPS for more complex models, such as DeiT-B, while improving Top-1 accuracy—a noteworthy exception to the typical trade-off where aggressive pruning often leads to performance loss. Additionally, both WDPruning [40] and NViT [42] achieved comparable parameter sizes (see Table 5) when applied to DeiT-T. However, NViT demonstrated a more substantial reduction in the number of parameters than other ViT models. While these results demonstrate notable progress in pruning ViTs, the training and finetuning costs remain a major limitation, particularly for deployment on resource-constrained edge devices. Therefore, training-efficient or finetuned free pruning techniques need more attention in the near future for efficient deployment on the edge. This necessitates a more precise estimation of parameter or block sensitivity using limited data, as well as a deeper exploration of the information embedded within hidden features during training and inference.

2.2. Knowledge distillation

Knowledge distillation (KD) is another model compression technique in machine learning where a smaller model (the “student”) is trained to reproduce the behavior of a larger model (the “teacher”). The purpose is to transfer the “knowledge” from the larger model to the smaller one, thereby reducing computational resources without significantly losing accuracy. Pruning is a direct way to reduce the complexity of the original model, whereas KD involves training a new, more compact model that is easy to deploy. By using KD during finetuning, the pruned model can benefit from the insights and information captured by the larger teacher model. KD helps to regain the original performance and compensate for the accuracy loss during the other compression techniques (e.g., pruning, quantization).

Table 4

GFLOPS reduction comparison of pruning techniques across different ViT models. ‘↓’ denotes the reduction rate from the baseline ViT model.

Algorithm	GFLOPS (↓)				
	DeiT-T	DeiT-S	DeiT-B	Swin-S	T2T-ViT-14
Baseline	1.3	4.6	17.6	8.7	4.8
VTP [38]	–	–	10.0 (↓ 43.2%)	–	–
WDPruning [40]	0.7 (↓ 46.2%)	2.6 (↓ 43.5%)	9.9 (↓ 43.8%)	6.3 (↓ 27.6%)	–
MD [51]	–	2.9 (↓ 37.0%)	11.2 (↓ 36.4%)	–	2.9 (↓ 39.6%)
SPViT [37]	–	–	–	6.4 (↓ 26.4%)	–
PS-ViT [39]	0.7 (↓ 46.2%)	2.6 (↓ 43.5%)	9.8 (↓ 44.3%)	–	3.1 (↓ 35.4%)
UVC [41]	0.5 (↓ 61.5%)	2.3 (↓ 50.0%)	8.0 (↓ 54.5%)	–	2.1 (↓ 56.0%)
NViT [42]	1.3 (↓ 0.0%)	4.2 (↓ 8.7%)	6.8 (↓ 61.4%)	6.2 (↓ 28.7%)	–
SAViT [47]	0.9 (↓ 30.8%)	3.1 (↓ 32.6%)	10.6 (↓ 39.8%)	–	–
S ² ViTE [43]	0.9 (↓ 30.8%)	3.1 (↓ 32.6%)	11.8 (↓ 33.0%)	–	–
VTC-LFC [44]	0.76 (↓ 41.7%)	2.4 (↓ 47.1%)	8.03 (↓ 54.4%)	–	–
CP-ViT [36]	0.74 (↓ 43.3%)	2.7 (↓ 42.2%)	10.28 (↓ 41.6%)	–	–

Table 5

Parameter (Million) reduction comparison of pruning techniques across different ViT models. ‘↓’ denotes parameter reduction rate from the baseline ViT model.

Algorithm	Params (M ↓)			
	DeiT-T	DeiT-S	DeiT-B	Swin-S
Baseline	5.6	22	86	50
VTP [38]	–	–	47.3 (↓ 45.0%)	–
WDPruning [40]	3.5 (↓ 37.5%)	13.3 (↓ 39.5%)	55.3 (↓ 35.7%)	32.8 (↓ 34.4%)
NViT [42]	3.5 (↓ 37.5%)	10.5 (↓ 52.3%)	17.0 (↓ 80.3%)	15.0 (↓ 70.0%)
SAViT [47]	4.2 (↓ 25.0%)	14.7 (↓ 33.2%)	51.9 (↓ 39.6%)	–
S ² ViTE [43]	4.2 (↓ 25.0%)	14.6 (↓ 33.6%)	56.8 (↓ 34.0%)	–
VTC-LFC [44]	4.2 (↓ 25.0%)	15.3 (↓ 30.4%)	56.8 (↓ 34.0%)	–

2.2.1. KD techniques for vision transformer

Current KD techniques on ViT models can be broadly classified into three types: logit-based [55,56], relation-based [17,57], and feature-based [58–60] methods. Feature-based KD enables the student model to capture richer semantic representations by aligning intermediate features across hidden layers. Both logit-based and feature-based KD typically focus on transferring knowledge from individual data samples. In contrast, relation-based KD extends this idea by modeling relational knowledge across the whole dataset, aiming to capture inter-sample relationships. Fig. 6 illustrates a schematic overview of these KD types, providing a visual summary of their key differences. However, A detailed analysis of these three categories has also been conducted by Yang et al. [61]. In the following subsections, we provide a detailed discussion of the current proposed techniques on ViT models of each category.

Logit-based KD Logit-based KD (see Fig. 6(a)) focuses on learning knowledge from the last layer and aims to align the final predictions

between the teacher and student. Touvron et al. [46] pioneered a benchmark KD technique for ViTs by enabling training with substantially smaller datasets. The authors introduced a distillation token, an additional learnable vector used alongside the class token during training. Recently, TinyViT [55] highlighted that smaller ViTs could benefit from larger teacher models trained on extensive datasets, such as distilling the student model on ImageNet-21k and finetuning on ImageNet-1k. This approach reduced the computation significantly during the training of the student model. To optimize computational memory, TinyViT introduced a strategy that pre-stores data augmentation details and logits for large teacher models, reducing memory overhead. One problem with TinyViT is that teacher models still use large datasets. To solve this issue, DearKD [56] proposed the KD methods on self-generative data and used representational KD on intermediate features with response-based KD. The proposed paper used mean square error (MSE) distillation loss for hidden features, CE loss for hard label distillation, and intra-divergence distillation loss function to

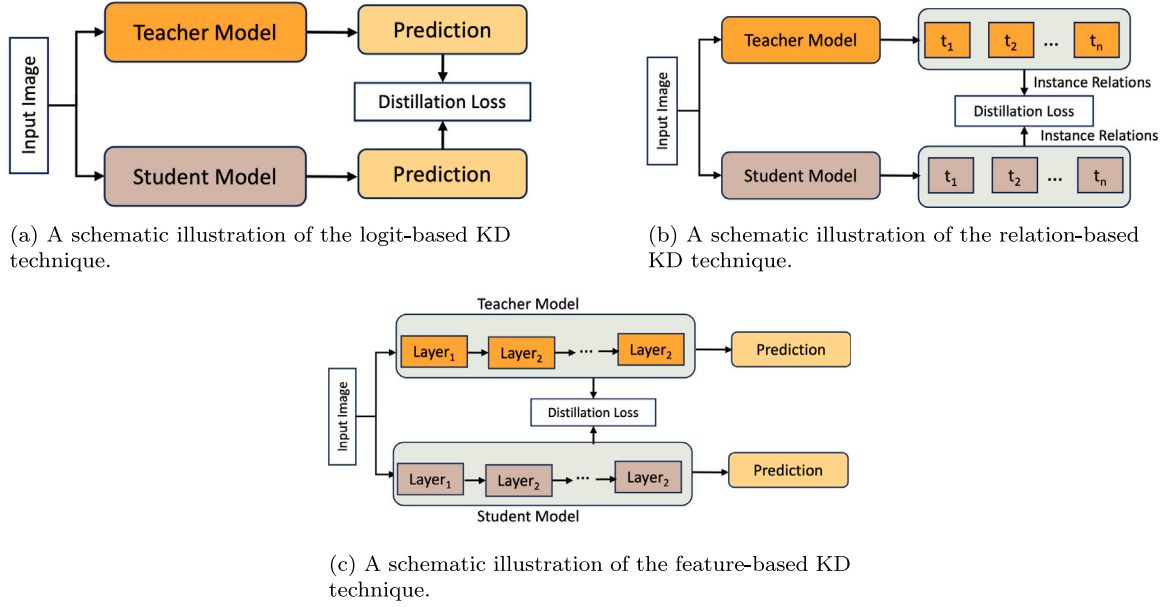


Fig. 6. An illustration depicting the three types of KD: logit-based, relation-based, and feature-based between teacher and student models.

Table 6

Results of KD techniques applied to ViT models for classification tasks. MSE refers to Mean Squared Error loss. '†' in the student model indicates the performance improvement over the baseline student model.

Type	Algorithm	Method	Loss function	Dataset	Teachers		Students	
					Models	Top-1 (%)	Models	Top-1 (%)
Relation	TAT [17]	One-to-all spatial matching KD	Vanilla distillation + L_2 loss	ImageNet-1k [45]	ResNet34	72.4%	ResNet18	72.1 († 2.0%)
	Co-advise [57]	Co-advised by lightweight teachers	Kull back divergence + Cross entropy loss	ImageNet-1k [45]	ResNet18	83.4%	CiT-Ti	88.0 († 1.5%)
Feature	Manifold [58]	Patch-level manifold space method	Manifold distillation loss	ImageNet-1k [45]	CaiT-S24	83.4%	DeiT-T	76.5 († 4.3%)
						83.4%	DeiT-S	82.2 († 2.3%)
					Swin-S	83.2%	Swin-T	82.2 († 1.0%)
	AProbe [60]	Probe & Knowledge distillation	Probe distillation + Cross-entropy loss	CIFAR-100 [52]	DeiT-XS	76.30%	DeiT-T	71.8 († 6.36%)
				CIFAR-10 [52]		96.65%	DeiT-T	93.9 († 7.64%)
				MNIST [62]		99.39%	DeiT-T	99.1 († 0.01%)
Logit	MiniViT [59]	Weight distillation	Self-attention + Hidden-state + Prediction loss	ImageNet-1k [45]	RegNet- 16GF	82.9%	DeiT-B	83.2 († 1.4%)
	TinyViT [55]	Reusing teacher predictions & data augmentation	Cross entropy loss	ImageNet-1k [45]	CLIP- ViT-L	84.8%	Swin-T	83.4 († 2.2%)
							DeiT-S	82.0 († 2.1%)
	DearKD [56]	Self-generative data	MSE distillation + Cross entropy+ Intra -divergence loss	ImageNet-1k [45]	ResNet- 101	77.37%	DeiT-Ti	71.2 (↓ 1.0%)

Co-advise [57]: Transformer-Ti is denoted as CiT-Ti in our table.

calculate the loss. It was noteworthy that DearKD surpassed the performance of the baseline ViT model trained with ImageNet-21K datasets, even though it only used 50% of the data. However, logit-based KD only transfers output-level predictions, neglecting intermediate representations such as features or attention. This shallow supervision limits its effectiveness, especially when the student and teacher architectures differ significantly.

Feature-based KD As we mentioned above, logit-based KD neglects intermediate-level supervision for complete guidance. To solve this problem, feature-based KD (see Fig. 6(c)) focuses on exploring intermediate feature information, such as feature maps and their refined

information. Hao et al. [58] introduced a fine-grained manifold distillation technique, which considered ViT as feature projectors that map image patches into a sequence of high-dimensional manifold spaces. The authors then teach the student layers to generate output features having the same patch-level manifold structure as the teacher layer for manually selected teacher-student layers. These output features are normalized and reshaped to compute a manifold relation map, a representation of the manifold structure of the features. However, the manifold relation map computation is resource-consuming and needs to be simplified. To address this computational issue, the authors proposed a decomposition strategy that separates the manifold relation map into three components: intra-image, inter-image, and

Table 7
Results of KD techniques applied to ViT models for object detection tasks.

Algorithm	Pretrained dataset	Dataset	Students			
			Without KD		With KD	
			Models	AP ^{box}	Models	AP ^{box}
Manifold [58]	ImageNet-1k [45]	COCO-2017	Swin-T	43.7	Swin-T	44.7 (↑1.0)
MiniViT [59]	–	COCO-2017	Swin-T	48.1	Swin-T	48.6 (↑0.5)

randomly sampled relation maps. A dedicated manifold distillation loss (MD Loss) is then applied by aggregating the losses computed from each decoupled manifold relation map. However, the proposed method only relies on prediction-level distillation. To consider the other level (e.g., attention level), Minivit [59] considered both attention level and hidden-state distillation, guiding a small model to replicate the behavior of the teacher model. The key idea of the proposed method is to multiplex the weights of consecutive transformer blocks by sharing the weights of the MSA and MLP layers. Besides, weight distillation over self-attention transfers knowledge from a large teacher model to a compact student. To ensure effective knowledge transfer, MiniViT introduced a multi-level distillation loss that combines three components: (i) a prediction-logit loss that aligns the student's output with the teacher's softened logits, (ii) an attention distillation loss that encourages similarity in self-attention, and (iii) a hidden-state distillation loss that preserves structural relationships. Furthermore, Wang et al. [60] introduced another feature-based KD technique for ViT named attention probes (AProbes), which streamline ViTs using unlabeled data. The proposed method operates in two stages: first, an attention-based probing mechanism identifies informative samples from diverse unlabeled datasets. Next, a probe-guided distillation process transfers knowledge to the student model by aligning the final predictions and intermediate features with the teacher. Both stages are achieved through cross-entropy loss and a dedicated probe distillation loss. However, feature-based KD methods only focus on individual sample representations despite leveraging intermediate features and often overlook the underlying dependencies across layers. This limitation restricts their ability to capture the holistic knowledge embedded in the teacher model fully. To overcome this limitation, relation-based KD aims to model and transfer relational information.

Relation-based KD Relation-based KD (see Fig. 6(b)) is often modeled as an instance graph, where nodes represent sample embeddings and edges are weighted by similarity metrics. One notable example is Co-advise [57], which introduced a multi-teacher framework where lightweight teacher models with diverse inductive biases (e.g., CNNs) collaboratively guide a student ViT model. The key insight is that the inductive bias of a teacher has a more significant impact on student performance than their standalone accuracy. By distilling complementary inductive biases, the student gains a more comprehensive representation of the data. A token-level alignment mechanism is also proposed to match the inductive bias of student tokens with their respective teachers. Another method, target-aware transformer (TAT) [17], addressed spatial misalignment in distillation. It transferred each pixel in the teacher feature map to all spatial locations in the student using a transformer-based similarity mechanism, enabling fine-grained relational alignment. While Co-advise emphasizes architectural diversity and inductive bias, and TAT focuses on spatial adaptability, both exemplify the strength of relation-based KD in enhancing knowledge transfer beyond isolated feature matching.

2.2.2. Discussion

A key strength of ViT models lies in their scalability to high parametric complexity; however, this demands significant computational resources and incurs substantial costs. KD offers a way to transfer knowledge into more compact student models, yet challenges remain, particularly in the vision domain.

From the results in Tables 6–8, we observe a consistent trend: KD techniques can significantly improve the performance of student ViT models. In Table 6, relation-based KD techniques such as Co-advise [57] demonstrate noticeable top-1 accuracy improvements on the ImageNet-1k dataset, emphasizing the effectiveness of capturing inter-sample relationships. Feature-based techniques like MiniViT [59] achieve considerably higher top-1 accuracy than manifold distillation [58] by leveraging intermediate-level information such as attention and hidden states. In contrast, while logit-based methods offer more straightforward supervision through output alignment, their performance improvements tend to be more subtle and, in some cases (e.g., DearKD [56]) demonstrate slight accuracy degradation.

In object detection tasks (see Table 7), KD techniques yield consistent gains in AP^{box}, indicating successful knowledge transfer in detection pipelines. Although the improvements are smaller than in classification tasks, they still affirm the benefit of distillation in dense prediction contexts. For segmentation tasks (see Table 8), the application of KD leads to substantial mean intersection over union (mIoU) improvements. Additionally, we observe that the majority of existing KD approaches for ViT models have been proposed for classification tasks. At the same time, significantly fewer works have explored their applicability to object detection and segmentation tasks. This highlights a promising avenue for future research, particularly as vision-driven domains such as autonomous driving and medical imaging continue to gain momentum and demand high-performance yet efficient models.

2.3. Quantization

Quantization is used to reduce the bit-width of the data flowing through a NN model. So, it is used primarily for memory saving, faster inference times, and simplifying the operations for compute acceleration. That makes quantization essential for deploying NN on edge devices with limited computational capabilities. Quantization can be applied to different aspects of techniques. We organize our quantization discussion into two subsections. Firstly, we categorize different quantization techniques applied according to different aspects in Section 2.3.1. Lastly, we discuss different quantization techniques in ViT in Section 2.3.2.

2.3.1. Taxonomy of quantization methods

Table 9 categorizes quantization techniques based on their role in the quantization pipeline, including when quantization is applied (timing), how it is performed (scheme), how scaling factors are chosen (calibration), the granularity of quantization, and precision Strategies strategies.

Quantization Schemes Quantization schemes are broadly categorized into uniform and non-uniform techniques. Any weight or activation values in an NN can follow either a uniform or non-uniform distribution. Uniform quantization maps continuous weight and activation values to discrete levels with equal spacing between quantized values. However, non-uniform quantization sets different quantization steps for different parts of the data based on their distribution and importance to the final performance of the model. Non-uniform quantization can improve accuracy but is often more complex to implement in hardware.

Quantization Approaches Quantization approaches can be broadly classified based on whether they require retraining. Quantization-aware

Table 8

Results of KD techniques applied to ViT models for segmentation tasks.

Algorithm	Dataset	Metrics	Teachers		Students		Proposed	
			Models	Result	Models	Result	Models	Result
Manifold [58]	ADE20K [63]	mIoU	Swin-S + UPerNet	47.64	Swin-S + UPerNet	44.51	Swin-T + UPerNet	45.66 ($\uparrow 2.58\%$)
TAT [17]	COCO-Stuff10k [64]	mIoU	ResNet18	33.10	ResNet18	26.33	ResNet18	28.75 ($\uparrow 9.09\%$)
	Pascal VOC [65]	mIoU	ResNet18	78.43	ResNet18	72.07	ResNet18	75.76 ($\uparrow 9.28\%$)

Table 9

Categorization of quantization techniques across five key aspects: schemes, approaches, calibration methods, granularity levels, and deployment strategies.

Aspects of quantization	Techniques	Advantages	Limitation
Quantization schemes	Uniform [66,67]	Simple implementation & fast inference	Inflexible to data distribution
	Non-uniform [68,69]	High accuracy & Efficient bandwidth	Complex implementation & data-dependent
Quantization approaches	PTQ [14,70]	No retraining required & low overhead	Accuracy drop
	QAT [71–73]	Preserves accuracy & robust	Requires retraining & computational intensive
Calibration methods	Static [74,75]	Lightweight	Accuracy drop
	Dynamic [76,77]	Adaptive to input variations	Runtime overhead
Granularity	Layer-wise [78–80]	Simple implementation	Suboptimal precision for sensitive layers
	Channel-wise [78,81,82]	Fine-grained control	Computational intensive
Precision	Mixed-Precision [79,83–85]	Balances accuracy & efficiency	Not hardware-friendly
	Hardware-aware [85,86]	Hardware specific optimization	Platform-specific

training (QAT) incorporates quantization into both forward and backward passes during training, allowing the model to adapt to lower-precision representations. However, QAT is resource-intensive due to the need for retraining. In contrast, post-training quantization (PTQ) is a more efficient approach that applies quantization after a model has been fully trained in floating point (FP) precision, reducing the precision of weights and activations without additional training. While PTQ is less resource-intensive, it typically results in a higher accuracy drop compared to QAT [87].

Calibration Methods Calibration is a needed process of determining the appropriate scaling factors during finetuning methods that map the continuous range of FP values to discrete integer values. Calibration mainly ensures that the range of the quantized values matches the range of the original FP values as closely as possible. There are two types when choosing the range. One is dynamic quantization, and the other is static quantization. The weights are quantized statically in dynamic quantization, but activations are quantized dynamically at runtime. Static quantization is quantized post-training. Unlike dynamic quantization, static quantization applies to weights and activations before deploying the model.

Granularity Another aspect of quantization techniques is the granularity of the clipping range of an NN. Layer-wise quantization is one of the granularity techniques where all weights and activations within a layer are quantized using the same scale. Channel-wise quantization, also referred to as per-channel quantization, is another granularity technique applied during the quantization of NN. Different scaling factors are computed for each channel of the weights in channel-wise quantization, meaning different layers can use different quantization parameters.

Precision Strategies The precision strategies can be divided into three types: fixed precision, mixed-precision, and hardware-aware precision. In fixed-precision quantization, the entire model is quantized using a uniform bit-width (e.g., INT8), simplifying implementation but potentially leading to suboptimal accuracy for layers more sensitive to quantization. To address this limitation, mixed-precision during quantization has been introduced as a more flexible alternative.. Mixed-precision uses different parts (e.g., channels, layers) of the model that

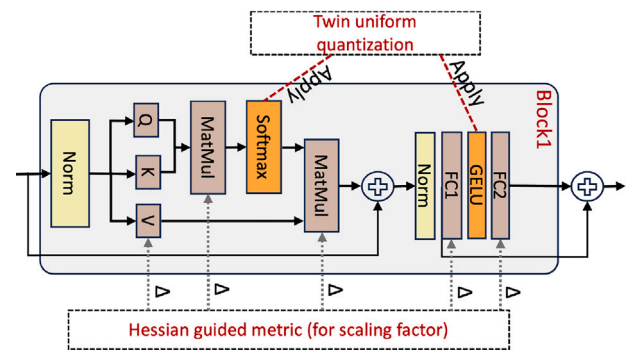


Fig. 7. An overview of the quantization framework in PTQ4ViT [66], which integrates twin uniform quantization and a Hessian-guided metric for determining scaling factors. Building on this, APQ-ViT [88] addresses the critical quantization error caused by Hessian-based scaling through a block-wise evaluation.

are quantized to different numerical precisions. Unlike fixed precision, where the entire model is quantized to the same bit-width (like INT8), mixed-precision involves carefully choosing the bit-width for each layer or even each channel within a layer based on their sensitivity and contribution to the final performance of the model. Despite these advantages, mixed-precision quantization is often not hardware-friendly due to the overhead of managing multiple precision levels within the same model. Hardware-aware quantization has emerged as a solution that tailors a neural network's precision reduction process according to the hardware requirement. This technique optimizes the model for the target hardware by adjusting the quantization parameters to match the hardware's operations [85], such as latency and throughput.

2.3.2. Quantization techniques for vision transformer

Applying quantization to ViT models is quite new in the neural network sector. ViT consists of multiple layers, including a self-attention mechanism and a feedforward neural network. As data passes through these layers, different layers learn to focus on different features of the input data. Applying quantization to ViT can be challenging due to its

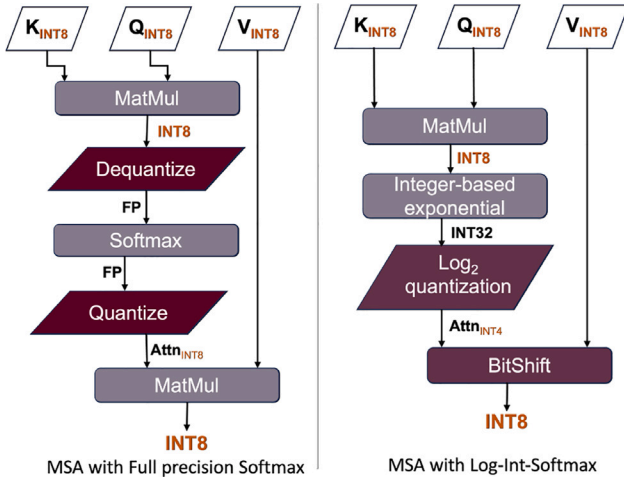


Fig. 8. Comparison of using full precision Softmax and log-int-softmax in quantized MSA inference in FQ-ViT [14].

complexity. Additionally, the loss from quantization can significantly impact the self-attention mechanisms, potentially reducing the model's overall performance. Quantization methods on ViT can be broadly categorized into two main approaches based on their reliance on training or finetuning: PTQ and QAT.

2.3.2.1. PTQ techniques for vision transformer. In the current scenario, PTQ is widely used for ViT because it offers an efficient way to meet the computational requirements on edge without additional training or finetuning, making it ideal for resource-constrained deployments. PTQ works can be divided into two categories [89,90]: statistic-based PTQ and learning-based PTQ. Statistic-based PTQ methods focus on finding optimal quantization parameters to reduce quantization errors. In contrast, learning-based PTQ methods involve finetuning both the model weights and quantization parameters for improved performance [91].

Statistic-Based PTQ methods Most of the current PTQ works on ViT follow. **statistic-based methods.** As illustrated in Fig. 7, PTQ4ViT [66] addressed two key issues with base-PTQ for ViTs: (1) Unbalanced distributions after softmax and asymmetric distributions after GELU. (2) Traditional metrics are ineffective for determining quantization parameters. To solve the first problem, the authors proposed twin uniform quantization, which quantized values into two separate ranges. Additionally, to solve the second problem, they introduced a Hessian-guided metric for improved accuracy instead of MSE and cosine distance. While effective in general cases, the Hessian-guided metric struggles in ultra low-bit scenarios (e.g., 2-bit or 4-bit), often missing key quantization errors. APQ-ViT, proposed by Ding et al. [88], addresses this limitation through refined error modeling. The authors first proposed a unified bottom-elimination block-wise calibration for extremely low-bit representation. This block-wise calibration scheme enables a more precise evaluation of quantization errors by focusing on block-level disturbances that impact the final output. For the second challenge, they observed the “Matthew-effect” in the softmax distribution, where smaller values shrink further, and larger values dominate. Existing quantizers often overlook this issue, leading to information loss from the larger values. To address this, the authors proposed Matthew-effect preserving quantization (MPQ) for softmax to retain its power-law characteristics and ensure more balanced information retention during quantization.

Quantization Error with Noisy Bias Liu et al. proposed NoisyQuant [92], where they focused on adding a noisy bias to each layer to modify the input activation distribution to reduce the quantization error. The noisy bias is a single vector sampled from a uniform distribution. The

authors removed the impact of noisy bias after the activation-weight multiplication in the linear layer with a denoising bias so that the method could retrieve the correct output. Surprisingly, the experiment showed that adding a noisy bias improved top-1 accuracy compared to the PTQ4ViT [66] on the ViT-B, DeiT-B, and Swin-S model.

Integer-only PTQ Methods Unlike earlier PTQ approaches that rely on FP32 precision (via dequantization) during inference, recent methods have introduced fully integer-only quantization for ViTs [14,15,93]. These advancements eliminate floating-point dependencies, reduce computational overhead, and significantly enhance the suitability of ViTs for deployment on resource-constrained edge devices. Lin et al. first introduced fully quantized PTQ techniques named FQ-ViT [14], leveraging the power-of-two factor (PTF) method to minimize performance loss and inference complexity. To solve the non-uniform distribution in attention maps and avoid dequantizing to FP32 before softmax, they proposed that log-int-softmax (LIS) be replaced by softmax. Additionally, the authors further streamlined inference by employing 4-bit quantization combined with bit-shift operations. Fig. 8 (left) illustrates the traditional approach where the traditional approach dequantizes INT8 query (Q) and key (K) matrices to FP before softmax, re-quantizing afterward for attention computations. In contrast, the proposed method (Fig. 8 (right)) introduced matrix multiplication followed by integer-based exponential (i-exp). The authors then utilized Log_2 quantization scale in the softmax function and converted the MatMul to BitShift between the quantized attention map and values (V). This integer-only workflow, including LIS in INT4 format, significantly reduces memory usage while maintaining precision. Extended from FQ-ViT, Li et al. introduced I-ViT [93], the first integer-only PTQ framework for ViT, enabling inference entirely with integer arithmetic and bit-shifting, eliminating FP operations. In this framework, the authors utilized an integer-only pipeline named dyadic anthemic for non-linear functions such as dense layers. In contrast, non-linear functions, including softmax, GELU, and LayerNorm, were approximated with lightweight integer-based methods. The key contribution of this work is that Shiftmax and ShiftGEU replicated the behavior of their FP counterparts using integer bit-shifting. Despite I-ViT's reduction in bit-precision for parameters and its emphasis on integer-only inference, it retained its accuracy. For example, when I-ViT applied to DeiT-B, it achieved 81.74% top-1 accuracy with 8-bit integer-only inference, outperforming I-BERT [94] by 0.95% (see Table 10).

Algorithm 1 Fast Progressive Combining Search (FPCS)

- 1: **Input:** Coefficients x, y, z_1, z_2, k, p ; a pretrained full-precision model; a set of calibration data D_{calib} ; and the l th layer to be quantized ϕ_l .
- 2: **Output:** Quantization hyperparameters a^*, b^*
- 3: **# The initialization step:**
- 4: Generate the raw input X_l and output O_l by ϕ_l based on D_{calib} , and compute the percentiles $pct_{0.1}, pct_{0.9}$ and pct_1 by [78].
- 5: Compute the uniform partition of the first and second hyperparameters as $\mathcal{A} = \{pct_{0.1} + i \cdot \tau_A | i = 0, \dots, x\}$ and $\mathcal{B} = \{pct_{0.9} + j \cdot \tau_B | j = 0, \dots, y\}$ with $\tau_A = (pct_1 - pct_{0.1})/x$ and $\tau_B = (pct_1 - pct_{0.9})/y$.
- 6: Generate the candidate set C_0 as the Cartesian product of $\mathcal{A}$ and $\mathcal{B}$: $C_0 = \mathcal{A} \times \mathcal{B}$.
- 7: **# The progressive searching step:**
- 8: **for** $i = 0, \dots, p$ **do**
- 9: **# The coarse searching step:**
- 10: Construct the subset $C' \subset C_i$ by selecting the partitions that have the top- k smallest quantization loss.
- 11: **# The expanding step:**
- 12: Update the intervals for fine partitions: $\tau_A := \tau_A / (2 \cdot z_1)$, $\tau_B := \tau_B / (2 \cdot z_2)$.
- 13: Update the candidate set with fine partitions: $C_{i+1} = \{(a + i \cdot \tau_A, b + j \cdot \tau_B) | (a, b) \in C', i = -z_1, \dots, z_1; j = -z_2, \dots, z_2\}$.
- 14: **end for**
- 15: **return** $(a^*, b^*) \in C_p$ with the smallest quantization loss.

Table 10

Results of different post-training quantization(classification) techniques proposed for ViTs. **MP** denotes mixed precision; **W-bit** refers to weight bit-widths and **A-bit** refers to activation bit-widths. Here, Baseline refers to the closest comparable results for classification tasks.

Algorithm	Code	Key point	Backbone	Dataset	Results						
					Baseline	W-bit	A-bit	Top-1 accuracy			
PTQ [70]	✗	Introduce a mixed-precision weights strategy	ViT-B	CIFAR-10	Percentile	6 MP 8 MP	6 MP 8 MP	96.83 (+3.35) 97.79 (+2.03)			
				CIFAR-100	Percentile	6 MP 8 MP	6 MP 8 MP	83.99 (+3.15) 85.76 (+2.48)			
				ImageNet-1k [45]	Percentile	6 MP 8 MP	6 MP 8 MP	75.26 (+3.68) 76.98 (+2.88)			
				DeiT-B	ImageNet-1k [45]	Bit-Split	6 MP	6 MP	74.58 (+0.54)		
			PTQ4ViT [66]	✓	Twin-uniform quantization with Hessian-guided calibration	ViT-B DeiT-B Swin-B	ImageNet-1k [45]	Base-PTQ	8 8 8	8 8 8	85.82 (+0.52) 82.97 (+.64) 86.39 (+0.23)
APQ-ViT [88]	✗	Propose a Matthew-effect preserving scheme for low bit quantization	ViT-B	ImageNet-1k [45]	PTQ4ViT [66]	6 4	6 4	82.21 (+0.56) 41.41 (+10.72)			
						Swin-B/384	6 4	6 4	85.60 (+0.16) 80.84 (+2.0)		
			DeiT-B				6 4	6 4	80.42 (+0.17) 67.48 (+3.09)		
							NoisyQuant [92]	✓	Add a fixed noisy factor to the data distribution	ViT-B	6 8
			DeiT-B			6 8					6 8
Swin-S	6 8	6 8		84.57 (+1.22) 85.11 (+0.32)							
FQ-ViT [14]	✓	Power-of-two for LayerNorm & replace Softmax with Log-Int-Softmax	DeiT-T	ImageNet-1k [45]	Percentile	8	8	71.61 (+0.14)			
			DeiT-S			8	8	79.17 (+2.6)			
			DeiT-B			8	8	81.20 (+1.83)			
			Swin-B			8	8	82.97 (+42.04)			
I-ViT [93]	✓	Integer-only inference	ViT-B	ImageNet-1k [45]	I-BERT [94]	–	–	84.76 (+1.06)			
			DeiT-B			–	–	81.74 (+0.95)			
			Swin-S			–	–	83.01 (+1.15)			
RepQ-ViT [15]	✓	log _{√2} reparameterized to log ₂ during quantization & inference	ViT-B	ImageNet-1k [45]	APQ-ViT [88]	4	4	83.62 (+1.41)			
			DeiT-B			4	4	81.27 (+0.85)			
			Swin-S			4	4	82.79 (+0.12)			
MPTQ-ViT [95]	✗	Introduce SmoothQuant [96] with bias term	ViT-B	ImageNet-1k [45]	TSPTQ-ViT [97]	6	6	82.70 (+0.41)			
			DeiT-B			6	6	81.25 (+0.64)			
LRP-QViT [80]	✗	Importance-based layer bit precision	ViT-B	ImageNet-1k [45]	RepQ-ViT [15]	6 MP	6 MP	83.87 (+0.25)			
			DeiT-B			6 MP	6 MP	81.44 (+0.17)			
			Swin-S			6 MP	6 MP	82.86 (+0.07)			
ERQ [98]	✓	Introduced weight quantization error reduction metrics	ViT-B	ImageNet-1k [45]	AdaRound [99]	5	5	82.81 (+0.81)			
			DeiT-B			5	5	80.65 (+0.47)			
			Swin-S			5	5	82.44 (+0.32)			
Adalog [100]	✓	Proposed adaptive log-based quantization for post-Softmax & post GELU activations	ViT-B	ImageNet-1k [45]	RepQ-ViT [15]	6	6	84.80(+1.18)			
			DeiT-B			6	6	81.55 (+0.28)			
			Swin-S			6	6	83.19 (+0.40)			
CLAMP-ViT [101]	✓	Evolutionary search for optimal quantization parameters	DeiT-S	ImageNet-1k [45]	LRP-QViT [80]	6	6	79.43 (+0.40)			
			Swin-S			6	6	82.86 (+0.00)			

Decoupling Quantization for Edge-Efficient Inference The current studies consider quantizers and hardware standards to be always antagonistic, which is partially true. RepQ-ViT [15] decouples the quantization and inference process to explicitly bridge via scale reparameterization between these two steps. The authors applied channel-wise quantization for the post-LayerNorm activations to solve the inter-channel variations and $\log_{\sqrt{2}}$ quantization for the post-softmax activations. During inference, the proposed method efficiently reparameterized layer-wise $\log_2$ -based quantization to reduce the computational overhead associated with activation processing. Using integer-only quantization for all layers lessened the computational cost dramatically and made them highly suitable for edge devices.

Mixed-precision PTQ methods Recent studies have further advanced statistics-based PTQ for ViTs by incorporating **mixed-precision techniques**. Liu et al. [70] first explored a mixed precision PTQ scheme

for ViT architectures to reduce the memory and computational requirements. The authors estimated optimal low-bit quantization intervals for weights and inputs, used ranking loss to preserve self-attention order, and analyzed layer-wise quantization loss to study mixed precision using the L1-norm [102] of attention maps and outputs. Using calibration datasets from CIFAR-10 [52], ImageNet-1k [45], and COCO2017 [103], their method outperformed percentile-based techniques [78] by 3.35% on CIFAR-10 with ViT-B model. Recently, Tai et al. [95] and Ranjan et al. [80] both extended the mixed precision PTQ techniques on ViT. MPTQ-ViT [95] utilized the smoothQuant [96] with bias term (SQ-b) to address the asymmetry in activations, reducing clamping loss and improving quantization performance. The authors proposed a search-based scaling factor ratio (OPT-m) to determine the quantization parameters. Later, they incorporate SQ-b and OPT-m to propose greedy mixed precision PTQ techniques for ViT by allocating layer-wise bit-width. Additionally, Ranjan et al. [80] proposed LRP-QViT [80], an explainability-based approach by assessing each layer's contribution

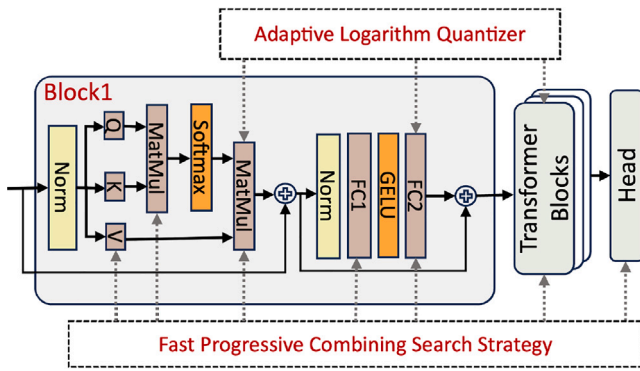


Fig. 9. The Overview of the AdaLog quantization method. AdaLog is applied to post-Softmax and post-GELU activations, with bias parameterization and a search strategy [100].

to the model's predictions, guiding the assignment of mixed-precision bit allocations based on layer importance. The authors also clipped the channel-wise quantization to eliminate the outliers from post-LayerNorm activations, mitigating severe inter-channel variations and enhancing quantization robustness. Zhong et al. proposed ERQ [98] to mitigate the error arising during quantization from weight and activation quantization separately. The authors introduced activation quantization error reduction to reduce the activation error, which is like a ridge regression problem. The authors also proposed weight quantization error reduction in an interactive approach by rounding directions of quantized weights coupled with a ridge regression solver.

Learning-Based PTQ Methods While most current PTQ methods for ViTs are statistic-based, there are only a few that utilize learning-based approaches. Existing PTQ methods for ViTs face challenges with inflexible quantization of post-softmax and post-GELU activations, which follow power-law-like distributions. One of the major challenges in learning-based approaches is efficiently determining the optimal quantization parameters, such as scaling factors. Adalog [100] introduced a fast progressive combining search (FPCS) strategy, which is designed to search the hyperparameter space effectively while maintaining linear complexity. The FPCS algorithm comprises two stages: initialization and progressive search. The latter performs a coarse search to select top candidate pairs, followed by an expansion step that locally refines the search. The complete procedure is outlined in Algorithm 1. On the other hand, CLAMP-ViT [101] leveraged contrastive learning in layer-wise evolutionary search to identify optimal quantization parameters.

As illustrated in Fig. 9, Adalog explored the adaptive $\log_2$ and $\log_{\sqrt{2}}$ to better align with the power-law distribution of activations while ensuring hardware-friendly quantization. The adaptive logarithm quantizers are applied to post-softmax and post-GELU activations through bias reparameterization. CLAMP-ViT [101] adopted a two-stage approach between data generation and model quantization. As illustrated in Fig. 10, stage 1 is to generate semantically rich and meaningful images. The stage 1 loss is used to adaptively refine the synthetic dataset in a self-supervised manner, forming a feedback loop with the generator. The refined synthetic images generated in Stage 1 are passed to Stage 2. Here, an evolutionary search is conducted on the quantized model to identify optimal layer-wise quantization parameters (e.g., bit-widths, scale factors). This search aims to minimize the Stage 2 loss, which reflects the fitness of quantized representations compared to the FP32 model. However, learning-based PTQ remains largely unexplored for low-bit quantization. Incorporating NAS to discover optimal quantization parameters automatically presents a promising future research direction instead of an evolutionary search.

2.3.2.2. QAT techniques for vision transformer. Compared to PTQ techniques, QAT methods for ViTs remain relatively underexplored. Existing QAT approaches can be broadly classified into two categories: leveraging KD to optimize the quantized model and standalone independent frameworks.

Leveraging KD in QAT Q-ViT [73] first proposed an information rectification module based on information theory to resolve the convergence issue during joint training of quantization. The authors then proposed distributed guided distillation by taking appropriate activities and utilizing the knowledge from similar matrices in distillation to perform the optimization perfectly. However, Q-ViT lacks other CV tasks, such as object detection. Another recent work, Q-DETR [84], is introduced to solve the information distortion problem. The authors explored the low-bit quantization of DETR and proposed a bi-level optimization framework based on the information bottleneck principle. However, Q-DETR failed to keep the attention activations less than 4 bits and resulted in mixed-precision quantization, which is hardware-inefficient in the current scenario. Both Q-ViT and Q-DETR explored the lightweight version of DETR apart from modifying the MSA. AQ-DETR [104] focused on solving the problem that exists for low bits of DETR in previous studies. The authors introduced an auxiliary query module and a layer-by-layer distillation module to reduce the quantization error between the quantized attention and its full-precision counterpart. All the previously discussed works are heavily dependent on the data. Li et al. [105] proposed PSAQ-ViT, aiming to achieve a data-free quantization framework by utilizing the property of KD. The authors introduced an adaptive teacher-student strategy enabling cyclic interaction between generated samples and the quantized model under the supervision of the full-precision model, significantly improving accuracy. The framework leverages task- and model-independent prior information, making it universal across various vision tasks such as classification and object detection.

Standalone QAT techniques Although most works utilized the KD in QAT works, there are limited standalone studies without KD. PackQViT [112] proposed activation-aware sub-8-bit QAT techniques for mobile devices. The authors leveraged $\log_2$ quantization or clamping to address the long-tailed distribution and outlier-aware training to handle the channel-wise outliers. Furthermore, the authors utilized int-2ⁿ-softmax, int-LayerNorm, and int-GELU to enable integer-only computation. They designed a SIMD-based 4-bit packed multiplier to achieve end-to-end ViT acceleration on mobile devices. Another recent study named QD-BEV [72] explored QAT on BEVFormer [117] by leveraging image and BEV features. The authors identified that applying quantization directly in BEV tasks makes the training unstable, which leads to performance degradation. The authors proposed view-guided distillation to stabilize the QAT by conducting a systematic analysis of quantizing BEV networks. This work will open a new direction for autonomous vehicle research, applying QAT to reduce computational costs.

2.3.3. Discussion

Tables 10 and 11 provide an overview of PTQ techniques for image classification and object detection on ViT models. A notable trend is the dominance of PTQ methods, with fewer studies exploring training-phase QAT. Table 10 highlights the top-1 accuracy improvements achieved by PTQ methods on ViT architectures for the classification tasks, while Table 11 summarizes the mean average precision (mAP) and AP^{box} improvement achieved from the baseline techniques for the object detection task. Moreover, Table 12 summarizes QAT techniques for image classification and object detection. Interestingly, most QAT methods for ViTs, such as those leveraging KD [73,84,104,105], focus on optimizing quantized models but lack experimental validation on edge devices. In contrast, approaches like PackQViT [112] have experimented with their QAT frameworks on mobile devices, pushing the boundaries of practical deployment. However, these quantization

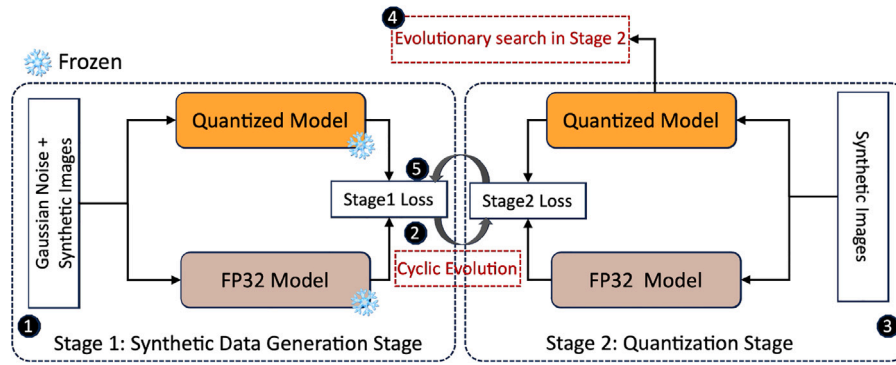


Fig. 10. The Overview of the Clamp-ViT [101]. Stage 1 corresponds to steps ①–②, Stage 2 corresponds to steps ③–④, and the iterative refinement process is denoted by step ⑤.

Table 11

Results of different quantization (object detection) techniques proposed for ViTs. The algorithms are experimented on COCO 2017 [106] datasets for object detection tasks. Here, Baseline refers to the closest comparable results for object detection. **MP** denotes mixed precision.

Algorithm	Backbone	Results				
		Baseline	W-bit	A-bit	mAP	AP ^{box}
PTQ [70]	DETR	Easyquant [107]	6 MP	6 MP	40.5(+1.5)	–
			8 MP	8 MP	41.7(+1.3)	–
APQ-ViT [88]	Mask-RCNN+Swin-T	PTQ4ViT [66]	6	6	–	45.4 (+39.6)
			4	4	–	23.7 (+16.8)
FQ-ViT [14]	Mask-RCNN+Swin-S	OMSE [108]	8	8	47.8(+5.2)	–
NoisyQuant [92]	DETR	PTQ [70]	8	8	41.4(+0.2)	–
RepQ-ViT [15]	Mask-RCNN+Swin-S	APQ-ViT [88]	6	6	–	47.8 (+0.1)
	Cascade Mask-RCNN+Swin-S		6	6	–	44.6 (+0.1)
LRP-QViT [80]	Mask-RCNN+Swin-S	RepQ-ViT [15]	6 MP	6 MP	–	48.1 (+0.3)
	Cascade Mask-RCNN+Swin-S		6 MP	6 MP	–	51.4 (+0.0)

Table 12

Results of different QAT techniques proposed for ViTs. The results of QD-BEV [72] are the mean average precision (mAP) rather than AP^{box}.

Algorithm	Code	Key point	Dataset	Results					
				Baseline	Model	W-bit	A-bit	Top-1(%)	AP ^{box}
Q-ViT [73]	✓	Enables 3-bit ViT QAT.	ImageNet-1k [45]	LSQ [109]	DeiT-B	4	4	83.0 (+2.1)	–
						2	2	74.2 (+3.9)	–
					Swin-S	4	4	84.4 (+1.9)	–
						2	2	76.9 (+4.5)	–
Q-DETR [84]	✓	KD to enhance quantized DETR's representation	PASCAL VOC [110]	LSQ [109]	TR50	2	2	–	50.7 (+8.1)
					SM-TR50	2	2	–	50.2 (+7.9)
			COCO 2017 [106]		TR50	4	4	–	39.4 (+6.1)
					SM-TR50	4	4	–	38.3 (+4.4)
AQ-DETR [104]	✗	QAT based on auxiliary queries for DETR	PASCAL VOC [110]	Q-DETR [84]	TR50	4	4	–	53.7 (+3.3)
					DE TR50	4	4	–	63.1 (+2.0)
			COCO 2017 [106]		TR50	4	4	–	40.2 (+2.8)
					DE TR50	4	4	–	44.1 (+3.4)
PSAQ-ViT2 [105]	✓	Data-free QAT using adaptive KD	ImageNet-1k [45]	PSAQ-ViT [111]	DeiT-B	8	8	81.5 (+2.4)	–
					Swin-S	8	8	82.1 (+5.5)	–
			COCO 2017 [106]	Standard V2	DC	8	8	–	44.8 (+3.)
					SC	8	8	–	50.9 (+0.6)
PackQViT [112]	✓	8-bit QAT for mobile devices	ImageNet-1k [45]	Q-ViT [73]	DeiT-B	8	8	82.9 (+0.5)	–
					Swin-S	8	8	84.1 (+0.5)	–
			COCO 2017 [106]	PTQ [70]	TR50	8	8	–	60.0 (–3.1)
QD-BEV [72]	✗	View-guided loss image and BEV features.	NuScenes [113]	BEV-DFQ [114]		8	8	–	40.6 (+2.2)
				BEV-HAWQv3 [115]	BEV	8	8	–	40.6 (+3.0)
				BEV-PACT [116]		8	8	–	40.6 (+3.2)

Note: Model names are shortened to save space in the table (SM TR50: SMCA-DeiT-R50).

TR50: DETR-R50; **DE:** Deformable; **DC:** DeiT-S with Cascade Mask R-CNN;

SC: Swin-S with Cascade Mask R-CNN; **SM:** SMCA; **BEV:** BEVFormer.

Table 13

The overview of popular software tools for deploying deep learning models on different hardware architectures.

Tool	Type	Key features				Supported edge devices		
		MO	MC	HLS	IR	FPGA	GPU	CPU
Xilinx Vivado design suite ¹	Toolkit	✗	✓	✓	✗	Xilinx FPGA	No	No
Intel Quartus Prime ²	Toolkit	✗	✓	✓	✗	Intel FPGA	No	No
OpenVINO ⁴	Toolkit	✓	✗	✗	✓	Intel FPGA	Yes	Yes
Vitis AI ³	Engine	✓	✓	✗	✗	Xilinx FPGA	No	No
FINN ⁶	Engine	✓	✓	✗	✓	Xilinx FPGA	No	No
TensorRT ⁷	Engine	✗	✗	✗	✓	No	NVIDIA GPU	No
ONNX Runtime ¹⁰	Engine	✗	✗	✗	✓	Yes	Yes	Yes
Hls4ml ⁵	Library	✗	✗	✓	✗	Yes	No	No
oneDNN ⁹	Library	✓	✗	✗	✓	No	Experimental	Intel CPU, ARM
NVIDIA Triton Inference Server ⁸	Inference Server	✗	✗	✗	✓	No	NVIDIA GPU	x86, ARM

MO: Model Optimization; **MC:** Model Compilation;**HLS:** High-Level Synthesis; **IR:** Inference Runtime

techniques have been broadly experimented on datasets like ImageNet-1k and COCO 2017, raising questions about their generalizability to specialized domains such as medical imaging or autonomous driving. This gap underscores the need for future research to explore versatile quantization strategies that cater to diverse application areas and resource-constrained edge devices.

It is important to note that the impact of model compression techniques such as pruning, quantization, and KD extends beyond theoretical efficiency. These methods directly influence how ViTs can be deployed on edge devices, especially when paired with hardware-aware acceleration strategies discussed in Section 4. Moreover, the practical effectiveness of these compression techniques often depends on the capabilities of software toolchains, which provide essential backend support for optimized deployment, as explored in Section 3.

3. Tools for efficient edge deployment

Efficient edge deployment of ViT requires a combination of software tools, evaluation tools, and advanced optimization techniques for different hardware architectures. Software tools streamline model deployment by providing optimized libraries and frameworks tailored for edge environments. Optimization techniques, such as memory optimization and pipeline parallelism, enhance performance by leveraging hardware-specific optimizations. Finally, heterogeneous platforms, including CPUs, GPUs, FPGAs, and custom accelerators, offer the flexibility to balance power, performance, and cost for various applications. In this section, we explore these essential pillars of edge deployment.

3.1. Software tools

Deploying deep learning models on heterogeneous platforms demands specialized software tools that bridge the gap between cutting-edge artificial intelligence (AI) research and real-world applications. These tools empower developers to optimize, accelerate, and seamlessly integrate AI models across different hardware architectures. Table 13 compares widely used deployment tools across four functional dimensions: model optimization (MO), model compilation (MC), high-level synthesis (HLS), and inference runtime (IR), with their hardware compatibility spanning FPGA, GPU, and CPU platforms.

As FPGAs offer highly parallel and reconfigurable hardware capabilities, deploying AI models on FPGAs requires specialized software tools for efficient hardware mapping, optimization, and deployment. Both Vivado Design Suite¹ (Xilinx) and the Quartus Prime Design Software² (Intel) offer advanced synthesis, converting high-level languages to

hardware description language (HDL) and preoptimized AI accelerators IP cores (such as Xilinx DPU or Intel AI Suite) that help to accelerate the inference task.

Vitis AI³ (for Xilinx) and OpenVINO⁴ (for Intel) provide support for partial model compression, hardware-optimized operator libraries, and integrated runtime environments. However, both frameworks are tightly coupled to their respective hardware ecosystems, which limits cross-platform portability. Each serves as a comprehensive deployment platform for FPGA-based applications, leveraging their corresponding hardware toolchains for synthesis, optimization, and implementation. Besides those two software tools, Hls4ml⁵ and FINN⁶ designed to explore deep neural network inference on FPGAs efficiently and swiftly. However, both these libraries are still in the experimental phase.

In the GPU domain, TensorRT⁷ uses as a popular inference optimizer and accelerating library for NVIDIA GPU-based edge devices. In a broader manner, Triton Inference Server⁸ enables the scalable, production-ready serving of multiple deep learning models on NVIDIA GPUs, x86, and Arm CPUs, integrating seamlessly with frameworks like TensorFlow, Keras, and PyTorch. However, neither supports FPGAs. On the other hand, oneDNN⁹ is a widely adopted, open-source performance library for deep learning applications, offering cross-platform support. It is highly optimized for Intel CPUs, integrated GPUs, and ARM-based processors, with ongoing experimental support for NVIDIA GPUs, AMD GPUs, and RISC-V architectures. ONNX Runtime¹⁰ stands out as a versatile, hardware-agnostic inference engine, supporting deployment across CPUs, GPUs, and FPGAs via backend execution providers. It balances broad compatibility with modular optimization and is often preferred in cross-platform production environments.

In practice, the choice of tool depends on the target device, optimization needs, and ecosystem. For example, TensorRT and Triton Inference Server are ideal for GPU deployments, Vitis AI excels in customized FPGA acceleration, while ONNX Runtime is well-suited for flexible and portable edge inference across multiple hardware types. In conclusion, unified toolkits for multiple hardware platforms might be an interesting research direction in the future.

³ Vitis AI. Retrieved January 18, 2025, from <https://www.xilinx.com/products/design-tools/vitis/vitis-ai.html>.

⁴ Openvinotoolkit. Retrieved January 18, 2025, from <https://github.com/openvinotoolkit/openvino>.

⁵ Hls4ml. Retrieved January 18, 2025, from <https://fastmachinelearning.org/hls4ml/index.html>.

⁶ FINN. Retrieved January 18, 2025, from <https://xilinx.github.io/finn/>.

⁷ TensorRT. Retrieved January 18, 2025, from <https://developer.nvidia.com/tensorrt>.

⁸ NVIDIA Triton Inference Server. Retrieved January 18, 2025, from <https://github.com/triton-inference-server/server>.

⁹ oneDNN. Retrieved January 18, 2025, from <https://github.com/oneapi-src/oneDNN>.

¹⁰ ONNX Runtime. Retrieved January 18, 2025, from <https://onnxruntime.ai/>.

¹ Vivado design suite. Retrieved January 18, 2025, from <https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html>.

² Intel Quartus Prime. Retrieved January 18, 2025, from <https://www.intel.com/content/www/us/en/products/details/fpga/development-tools/quartus-prime.html>.

3.2. Evaluation tools

Evaluating the performance of ViT acceleration techniques on edge platforms requires specialized tools and metrics to evaluate power consumption, energy efficiency, accuracy, and latency. Fortunately, most hardware vendors offer built-in tools and libraries to facilitate precise measurement of these key performance indicators.

3.2.1. Latency

Latency and frame per second (FPS) can be calculated as follows:

$$\text{FPS} = \frac{1}{\text{Latency}}$$

For GPU-based evaluations, PyTorch provides `torch.cuda.Event (enable_timing=True)` for GPU-based evaluations, which accurately measures latency during inference. On NVIDIA EdgeGPU platforms, the **TensorRT Profiler** offers a detailed latency breakdown for Jetson boards. For AMD FPGAs, the **Vitis AI Profiler**¹¹ enables profiling during deployment, ensuring optimized execution. Additionally, **Intel's OpenVINO benchmark tool**⁴ supports latency and throughput measurements across Intel CPUs and FPGAs, providing a standardized evaluation framework.

3.2.2. Power

Measuring power consumption is critical yet challenging in evaluating ViT acceleration techniques. Standard tools for general-purpose platforms (GPPs) like CPUs and GPUs include **Intel Power Gadget** for Intel CPUs and **NVIDIA-SMI** for NVIDIA GPUs. Power can be measured on edge GPU platforms, such as NVIDIA Jetson boards, using **tegraStats**, which provides real-time power monitoring, GPU utilization, and temperature. For FPGAs and ACAPs, AMD Xilinx offers **Xilinx Power Estimator (XPE)**¹² for power estimation based on hardware configurations, while **Vaitrace** enables runtime power profiling for FPGA and adaptive compute acceleration platforms (ACAP). These tools provide essential insights into power efficiency, thermal behavior, and overall performance trade-offs across edge hardware platforms.

3.2.3. Energy

Energy consumption in ViT acceleration can be estimated through throughput per joule (GOP/J) and FPS per watt (FPS/W). However, accurately measuring energy on general-purpose platforms (GPPs) is challenging due to background processes affecting power readings. In contrast, edge devices provide a more controlled environment where only one primary task is executed simultaneously, making energy estimation more reliable. Several tools facilitate energy measurement: **Xilinx Vivado Power Analyzer**¹ estimates energy efficiency for FPGAs by profiling dynamic power, while **RAPL** tracks CPU-level energy consumption on x86 architectures. Additionally, energy efficiency can be derived using power measurements combined with latency, enabling a deeper evaluation of acceleration techniques.

3.2.4. Resource utilization

Resource utilization is primarily analyzed in FPGA-based acceleration techniques to optimize hardware efficiency and minimize resource usage. Currently, Intel and AMD are two FPGA vendors. AMD offers **Vivado Design suite**¹ for synthesis evaluation of the FPGA before deployment. Similarly, Intel provides the **Quartus Prime**² software, which facilitates FPGA synthesis, resource utilization monitoring, and performance evaluation. Both vendors offer additional AI optimization frameworks – AMD's **Vitis AI**³ and Intel's **FPGA SDK for OpenCL (AOCL)** – to enhance the efficiency of ViT acceleration on FPGA platforms.

¹¹ Vitis AI Profiler. Retrieved February 10, 2025, from https://github.com/Xilinx/Vitis-AI/tree/master/examples/vai_profiler.

¹² Xilinx Power Estimator. Retrieved January 18, 2025, from <https://www.amd.com/en/products/adaptive-socs-and-fpgas/technologies/power-efficiency/power-estimator.html>.

3.3. Common optimization techniques

Memory Optimization Techniques like Huffman coding can be used to compress the weights. On-chip memory utilization efficiently uses the FPGA's Block RAMs (BRAMs) to store weights and intermediate feature maps, reducing the need for off-chip memory accesses, which can be slow and power-hungry.

Pipeline Parallelism The technique splits the model into stages and simultaneously processes different inputs at each stage, which helps in maximizing the throughput.

Loop Unrolling This FPGA-specific optimization involves unrolling loops in the FPGA design to speed up the processing. For instance, when performing matrix multiplications in the transformer layers.

Layer Fusion Layer fusion combines multiple layers into a single computational unit, reducing memory access between layers and improving the overall throughput.

Hardware-friendly Activation Functions Replace complex activation functions with simpler, hardware-friendly alternatives. For example, using piecewise linear approximations for non-linearities.

Optimized Matrix Operations ViT involves many matrix multiplications (in the attention mechanisms). Optimizing these matrix operations for FPGA leads to significant speed-ups. Techniques like systolic arrays or optimized linear algebra cores are employed.

Dynamic Precision In recent studies, some work uses mixed precision computations where certain parts of the model use lower precision (e.g., 8-bit). In comparison, other parts use higher precision (e.g., 16-bit or 32-bit). There are some works in which the authors introduced fixed point and PoT precision and optimally balanced accuracy and performance.

Although software frameworks enable optimized ViT execution with hardware-aware features, achieving real-time performance on edge devices often requires low-level, hardware-specific acceleration techniques. Section 4 explores dedicated acceleration methods for deploying ViTs on edge platforms.

4. Accelerating strategies for ViT on edge

This section explores acceleration strategies for non-linear operations, discusses SOTA ViT acceleration techniques, and provides a comprehensive performance analysis in terms of both hardware efficiency and accuracy.

4.1. Accelerating non-linear operations

ViT models in CV can mainly be split into two types of operations: linear and non-linear. Optimizing non-linear operations in quantized ViT is as crucial as optimizing linear operations. While low-bit computing units significantly reduce computational complexity and memory footprint, they are primarily designed for linear operations such as matrix multiplications and convolutions. However, non-linear functions, including softmax, GELU, and LayerNorm, are the essential components of ViT architectures yet largely unexplored in these low-bit computing environments. The lack of support for non-linear operations on hardware creates computational bottlenecks during the edge deployment, as non-linear functions often require FP32 operations. This results in latency and increases power and energy consumption, ultimately lessening the full benefits of quantization on edge deployment. Moreover, during inference on quantized ViT models on edge devices, frequent quantization and dequantization operations surrounding non-linear layers add further inefficiencies, slowing inference and reducing throughput [118]. Researchers developed integer-based approximations for non-linear operations to solve these issues, eliminating the need for frequent FP32 computations in those layers.

Table 14

Overview of ViT models utilizing integer approximations for non-linear operations to enhance inference efficiency and avoid dequantization.

Model	Experiment hardware	Non-linear operations			Retrain
		Softmax	GELU	LayerNorm	
FQ-ViT [14]	-	✓	✗	✓	✗
I-ViT [93]	RTX 2080 Ti GPU	✓	✓	✓	✓
EdgeKernel [119]	Apple A13 and M1 chips	✓	✓	✓	✗
PackQViT [112]	Snapdragon 870 SoC (Mobile Phone)	✓	✓	✓	✓
SOLE [120]	ASIC 28nm	✓	✓	✓	✗
SwiftTron [121]	ASIC 65 nm	✓	✓	✓	✗

As illustrated in Table 14, we introduce integer-only approximations for different hardware platforms to enhance ViT inference efficiency. As we discussed details in Section 2.3, FQ-ViT [14] utilized LIS for integer-only variants of the softmax that approximate the exponential component using a second-order polynomial coupled with log 2 quantization. For LayerNorm, the authors applied PTF to shift the quantized activations and later computed mean and variance using integer arithmetic. However, this approach seems hardware efficient; their methods of practical deployment on edge platforms are unclear. Additionally, I-ViT [93] calculates the square root in LayerNorm using an integer-based iterative method. The authors then introduced ShiftGELU, which used sigmoid-based approximations for GELU approximations. I-ViT utilized NVIDIA RTX 2080 Ti GPU as a hardware platform to evaluate their method.

PackQViT [112] is an extended version of the FQ-ViT concept that also leverages second-order polynomial approximations more straightforwardly, replacing the usual constant e with 2 in the softmax. Although PackQViT requires training, the simplification ensures no accuracy loss. EdgeKernel [119] addressed precision challenges in softmax computations by optimizing the selection of the bit shift parameter on Apple A13 and M1 chips, ensuring high accuracy while minimizing significant bit truncations. Additionally, it employs asymmetric quantization for LayerNorm inputs, converting them to a uint16 format to enhance computational efficiency while maintaining data integrity.

SOLE [120] and SwiftTron [121] both are utilized application-specific integrated circuit (ASIC) platforms. SOLE optimizes the software perspectives, introducing E2Softmax with log₂ quantization to avoid traditional FP32 precision in softmax layers. Additionally, SOLE designed a two-stage LayerNorm unit using PTF factors. However, SwiftTron focused on developing customized hardware for ASIC to efficiently use non-linear operations, even in FP32, which accounts for diverse scaling factors in performing correct computations.

4.2. Current accelerating techniques on ViT

Efficient acceleration techniques for seamless deployment on edge devices are essential where computational and energy constraints limit performance. Various techniques have been developed to optimize ViT execution, balancing throughput, latency, and energy efficiency while ensuring minimal loss in accuracy. These techniques can be broadly classified into SW-HW co-design and hardware-only acceleration. SW-HW co-design integrates algorithmic optimizations with hardware-aware modifications to ensure efficient deployment of ViTs on edge devices, enhancing throughput and energy efficiency. On the other hand, hardware-only approaches push efficiency even further by designing architectures specifically optimized for transformer workloads, utilizing systolic arrays, spatial computing, and near-memory processing to eliminate bottlenecks caused by data movement and external memory access. Additionally, alternative approaches, such as distributing multiple tiny edge devices by partitioning the model into submodels, offer promising directions to rethink ViT acceleration from a fundamentally different perspective.

4.2.1. Software optimization in SW-HW co-design

Recent advancements in software part in SW-HW co-design for ViT acceleration encompass a range of optimization techniques, including quantization-based acceleration [20,122–124], sparse and adaptive attention mechanisms [20,23], adding mixture of experts (MoE) layers [19], analyzing kernel profiling and execution scheduling [124], custom hardware softmax and LayerNorm to replace traditional softmax and LayerNorm [118,120].

Quantization-Based Software Acceleration VAQF [122] utilized binary quantization for weights and low-precision for the activations in the software part. VAQF automatically outputs the efficient quantization parameters based on the model structure and the expected frame per second (FPS) to meet the hardware specifications. The primary purpose of this work was to achieve high throughput on hardware while maintaining model accuracy. Moreover, Li et al. [123] used a mixed-scheme (fixed+PoT) ViT quantization algorithm that can fully leverage heterogeneous FPGA resources for a target FPS. Both frameworks utilized targeted FPS as their input to achieve maximum hardware efficiency during inference. Additionally, both VAQF and AutoViT-Acc [123] utilized PTQ methods as quantization. EQ-ViT [124] combined latency and accuracy requirements to decide the final quantization strategy, leveraging the QAT approach. Additionally, EQ-ViT implemented a profiling-based execution scheduler that dynamically allocates workloads across hardware accelerators.

Pruning-Based Software Acceleration ViT's self-attention has quadratic complexity concerning input sequence length, leading to high memory bandwidth consumption. Sparse attention mechanisms aim to reduce redundant computation by focusing on pruning. For instance, ViTCoD [23] efficiently employed structured pruning and polarized the attention maps to remove redundant attention scores, creating a more memory-efficient execution. Moreover, HeatViT [20] utilized image-adaptive token pruning, identifying and removing unimportant tokens before transformer blocks using a multi-token selector, dynamically reducing computational complexity. The proposed method is highly inspired by SP-ViT [37] and uses a similar token selector like SP-ViT.

Other Approaches Additionally, MoE improves model efficiency by activating only the most relevant expert networks per input instance, reducing unnecessary computations. One of the first studies named M³ViT [19] implemented MoE layers where a router dynamically selects the appropriate experts for processing. In this work, the authors conveyed training dynamics to balance large capacity and efficiency by selecting only a subset of experts using the MoE router. Beyond traditional quantization and attention optimizations, some co-design approaches target the computational bottlenecks of softmax and LayerNorm. SOLE [120] proposed E2Softmax and AllayerNorm, hardware-aware modifications for non-linear operations that replace FP32 with integer-only approximation. This integer-only computation improved the latency significantly during the inference.

4.2.2. Hardware optimization in SW-HW co-design

Hardware optimization plays a crucial role in the SW-HW co-design of ViTs, ensuring efficient execution across different accelerators. Key

Table 15

The overview of current accelerating techniques for ViT on edge devices. In the baseline models, B denotes Base; S denotes Small, and T denotes Tiny versions. Five types of hardware devices are used in existing SW-HW co-design frameworks: GPU (♣), EdgeGPU (▲), CPU (★), FPGA (♠), and AMD Versal Adaptive Compute Acceleration (ACAP) (◆).

Approaches	Framework	Retrain	Baseline models	Hardware device		Key optimization	
				Baseline	Experiment	Software	Hardware
SW-HW co-design	VAQF [122]	✗	DeiT-B/S/T	Intel i7-9800X* TITAN RTX♣	ZCU102♣	Quantization	Traditional resources
	Auto-ViT-Acc [123]	✗	DeiT-B/S/T	A100♣	ZCU102♣	Mixed quantization	Traditional resources
	HeatViT [20]	✓	DeiT-B/S/T	A100♣	ZCU102♣	Adaptive token pruning	Motivated from [123]
	EQ-ViT [124]	✓	DeiT-T	ZCU102♣ U250♣ AGX Orin* A100♣	VCK190◆ VEK280◆	Kernel-level profiling	Spatial & heterogeneous accelerators
	M ³ ViT [19]	✓	ViT-S/T	RTX 8000♣	ZCU102♣	Mixture of expert (MoE)	Computing MoE expert-by-expert
	ViTCoD [23]	✓	DeiT-B/S/T	Jetson Xavier* CPU★	ASIC (28 nm)	Prunes & polarizes the attention maps	On-chip encoder & decoder engines
	SOLE [120]	✗	DeiT-T	2080Ti♣	ASIC (28 nm)	E2Softmax & AILayerNorm	Custom hardware unit
Pure HW	ViA [21]	✗	Swin-T	Intel i7-5930X* V100♣	Alveo U50♣	–	Multi kernel parallelism with half mapping method
	ViTA [22]	✗	DeiT-B/S/T	ASIC (40 nm)	Zynq ZC7020♣	–	Head level pipeline & Inter-layer MLP
Other techniques	ED-ViT [125]	✓	ViT-B/S/T	–	Raspberry Pi-4B RTX 4090♣	–	Distributed edge devices for deploying submodels
	COSA Plus [126]	✗	ViT-B	RTX 3090♣ 6226R server CPU★	XCVU13P ◆	–	Systolic array with optimized dataflow

techniques include C++ based hardware descriptions, high-level synthesis (HLS), and accelerator bitstream generation [20,122,123]. Additionally, frameworks leverage AI engine (AIE) kernels [124] and custom hardware units (e.g., SOLE) to optimize execution for edge deployment.

ASIC-Based Hardware Acceleration ASIC-based accelerators have become a cornerstone in deploying deep learning models on edge devices, with the majority of advancements driven by leading technology companies. Notable examples include Google's TPU [127], IBM's TrueNorth [127], and NVIDIA's open-source NVDLA [128]. Academic research has also contributed with efficient designs such as Origami [129] and ZASCA [130], which are tailored for low-power convolutional inference.

Google's TPUs have released several versions [131]. TPUv1, released in 2016, featured a 256×256 systolic array of 8-bit fixed-point multipliers, delivering up to 92 TOPS at 75 W—well-suited for accelerating matrix-heavy operations like those found in ViT inference. TPUv2 and TPUv3 introduced support for FP operations and higher throughput. For resource-constrained environments, Google developed the EdgeTPU, a lightweight variant tailored for edge AI workloads.

As ViTs grow in popularity, their unique architectural demands have motivated the development of ViT-specific ASIC accelerators. Unlike CNN-based accelerators, ViT inference requires efficient handling of attention mechanisms and normalization layers. Recent efforts such as ViTCoD [23] and SOLE [120] addressed these challenges through SW-HW co-design strategies and have demonstrated promising ASIC implementations for ViT acceleration at the edge.

FPGA-Based Hardware Acceleration FPGA is the pioneer for hardware accelerating strategies because of its reconfigurable characteristics. VAQF [122] adopted the quantization schemes from the software part and applied them to the accelerator on the hardware side. Fig. 11 illustrates the overview of the VAQF framework. The accelerator's C++ description was synthesized using the Vivado HLS tool. Initial accelerator parameters focused on maximizing parallelism, but there were adjustments due to Vivado's placement or routing problems. Successful implementations produced a bitstream file for FPGA deployment. Moreover, as illustrated in Fig. 12, Auto-ViT Acc [123] first used the "FPGA Resource Utilization Modeling" module to give performance analysis

and calculate the FPS of the FPGA ViT accelerator in the software part. Inspired by VAQF, the authors implemented the FPGA accelerator using a C++ hardware description, synthesized through Vitis HLS to generate the final accelerator bitstream.

In ViTs, the computational bottleneck often arises from General Matrix Multiply (GEMM) operations, which form the core of self-attention and feed-forward layers. HeatViT [20] optimizes ViT execution by dynamically selecting tokens and loading each layer from off-chip DDR memory to on-chip buffers before processing via the GEMM engine. This approach, inspired by [123], minimizes redundant computations and improves memory efficiency. The proposed HeatViT addressed two main challenges of hardware implementation in their proposed architecture, as follows.

1. The GEMM loop tiling must be adjusted to factor in an extra dimension from multi-head parallelism.
2. ViTs have more non-linear operations than CNNs; these must be optimized for better quantization and efficient hardware execution while maintaining accuracy.

MoE Execution for Hardware Efficiency While MoE optimization techniques dynamically select experts, leading to an unpredictable computing pattern that makes hardware execution difficult. M³ViT [19] reordered computations to process tokens expert-by-expert rather than token-by-token, reducing irregular memory access and improving hardware parallelism. However, frequent off-chip memory accesses in MoE layers introduce a latency bottleneck. The authors utilized a ping-pong buffering technique for continuous processing without memory stalls, where one buffer fetches expert weights while another buffer performs computations. Additionally, the per-expert token queueing system groups tokens per expert, limiting the underutilization of compute units due to various expert demands.

Optimizing Non-Linear Operations with Custom Hardware Units Recent studies, such as EQ-ViT, separated matrix multiply (MM) and non-MM by efficiently mapping batch MM (BMM) and convolutions to AIE vector cores. Memory-bounded and non-linear operations are executed within the FPGA's programmable element. Additionally, EQ-ViT leveraged fine-grained pipeline execution to overlap computation

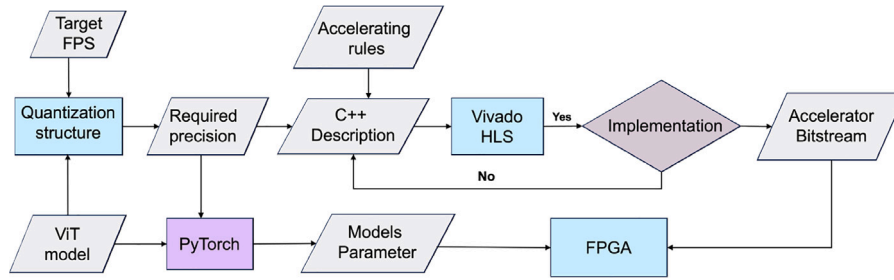


Fig. 11. Overflow of VAQF accelerator [122]. Using different colors in the architecture differentiates between types of processes within the overall workflow. The light gray boxes represent settings that are input/output to the process. The light blue boxes denote active processing steps or software tools within the workflow, like Vivado HLS. The lavender box signifies a platform/library used in the process, like “PyTorch”. The light purple box indicates decision-making points or critical stages in the architecture.

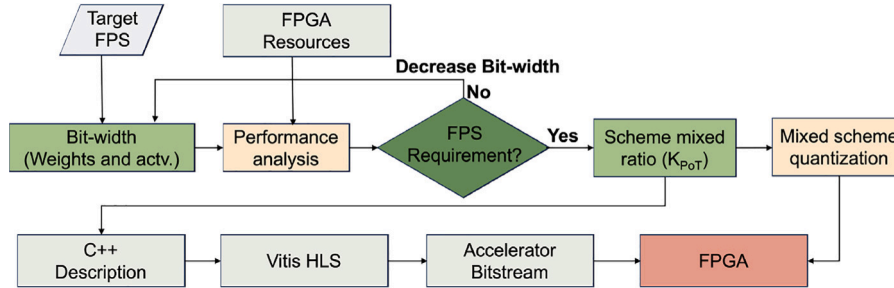


Fig. 12. The overview of Auto-ViT-Acc framework [123]. The “FPGA resource utilization modeling” was utilized for performance analysis and the estimated FPS rate for ViT accelerator with assigned bit-width for mixed schemes and lessening the bit-width until achieving the target FPS. The proposed mixed-scheme quantization then utilized mixed ratio (k_{pot}) results to implement on FPGA through “C++ Description for accelerator”, “Vitis HLS” and “Accelerator bitstream”.

with memory transfers, maximizing resource utilization. A key strength of this framework lies in its hardware mapping methodology, where execution is formulated as a mixed-integer programming (MIP) optimization problem, ensuring that latency and resource constraints are satisfied while maximizing throughput. Likewise, SOLE [124] designed separate units for the proposed E2Softmax and AILayerNorm to perform non-linear operations efficiently. The E2Softmax unit included Log2Exp and an approximate Log-based divider that is implemented in a LUT-free and multiplication-free manner. Additionally, the AILayerNorm unit operates in two stages: the first stage performs statistical calculations, while the second stage applies the affine transformation. Similar to M³ViT [19], SOLE also utilized ping pong buffer to pipeline the AILayerNorm unit.

Pure Hardware Accelerators Although most ViT acceleration techniques rely on SW-HW co-design – where software optimization plays a crucial role in achieving high efficiency and accuracy – the potential of pure hardware optimization has gained attention in recent studies. A recent study named ViA [21] addressed issues during data flow through the layers in ViT. The authors utilized a partitioning strategy to reduce the impact of data locality in the image and enhance the efficiency of computation and memory access. Additionally, by examining the computing flow of the ViT, the authors also utilized the half-layer mapping and throughput analysis to lessen the effects of path dependency due to the shortcut mechanism and to maximize the use of hardware resources for efficient transformer execution. The study developed two reuse processing engines with an internal stream, distinguishing them from previous overlaps or stream design patterns from the optimization strategies.

Moreover, ViTA [22] used two sets of MAC units to minimize the off-chip memory accesses. The first set of MAC units representing the hidden layers was broadcast to the second set of MAC units through a non-linear activation function. That broadcasting approach helped to compute the partial products corresponding to the output layer. The authors allocated these resources to maintain the pipeline technique as if the hidden layer value computations and the output layer partial product computation took equal time. This approach enabled the

integration of several mainstream ViT models by only adjusting the configuration.

Sparse Attention Optimization for ViTs Recent studies such as ViT-CoD [23] introduced a sparser engine to process the sparse attention metrics. The authors structured the multiply-accumulate (MAC) into the encoder and decoder MAC lines to optimize the matrix multiplications.

4.2.3. Other techniques

ED-ViT [125] utilized distributed workloads approaches to deploy the ViT model utilizing multiple tiny edge devices such as Raspberry Pi-4B. The authors partitioned the model into multiple submodels, mapping each submodel to a separate edge device. This distributed execution strategy enables ViTs to achieve efficiency comparable to single powerful edge accelerators like EdgeGPUs or FPGAs while leveraging cost-effective and scalable edge computing resources. Similarly, COSA Plus, proposed by Wang et al. [126], capitalized on high inherent parallelism within ViT models by implementing a runtime-configurable hybrid dataflow strategy. This method dynamically switches between weight-stationary and output-stationary dataflows in a systolic array, optimizing the computational efficiency for matrix multiplications within the attention mechanism. COSA Plus enhances processing element (PE) utilization by adapting data movement patterns to workload variations.

4.2.4. Discussion

From our observations, most of the SW-HW co-design proposed for FPGA or ACAP design due to their reconfigurable nature where the authors baselined GPU, CPU, or other FPGA platforms. Table 15 provides a comprehensive overview of the SOTA SW-HW co-design strategies for ViTs on various edge platforms. The analysis focuses on the hardware devices and optimization techniques. A key observation from this table is that different hardware platforms support distinct optimization levels. While GPUs and EdgeGPUs are widely used due to their parallel processing capabilities, FPGAs and ACAP platforms

Table 16

Compatibilities comparison of SOTA SW-HW co-design accelerating techniques. Here, GPP denotes general-purpose platforms such as CPU and GPU.

Framework	Baseline	Effort modulation	Prediction mechanism	Accuracy Top1 (%)	GPP compatible
ViTCoD [23]	DeiT-S	Constant	Norm Score	78.1	✗
HeatViT [20]	DeiT-S	Constant	Head level	79.1	✗
PIVOT [132]	DeiT-S	Input-aware	Entropy Metric	79.4	✓
VAQF [122]	DeiT-S	Hardware-driven	Hardware-aware	79.5	✗

Table 17

A comparative analysis of hardware performance for various ViT acceleration techniques, deployed on FPGA (♣) and ACAP (♠) platforms. Performance metrics include Frames Per Second (FPS), Energy Efficiency, and Throughput. The notation † represents FPS measured as images per second, while ⧫ denotes energy efficiency in GOP/J. Additionally, * indicates power consumption measured in Watt-Seconds (W S), providing deeper insights into the trade-offs between computation speed and energy usage.

Framework	Device	Precision	Frequency (MHz)	Resource utilization				FPS	Power (W)	Energy (FPS/W)	Throughput (GOPs)	Speedup (†)
				BRAM	DSP	KLUT	KFF					
VAQF [122]	ZCU102♣	W1A8	150	565.5	1564	143	110	24.8	8.7	2.85	861.2	–
ViA [21]	U50♣	FP16	300	1002	2420	258	257	–	39	7.94⧫	309.6	59.5x
ViTA [22]	ZC7020♣	INT8	150	–	–	–	–	8.7	0.88	3.13	–	2x
Auto-ViT-Acc [123]	ZCU102♣	W8A8+W4A8	150	–	1556	186	–	34.0	9.40	3.66	1181.5	–
HeatViT [20]	ZCU102♣	INT8	150	528.6	2066	161.4	101.8	11.4	54.8	4.83	–	4.89x
EQ-ViT [124]	VCK190♠	W4A8	–	16♠	28♠	6.5♠	–	10695†	–	224.7	–	–
Zhang et al. [133]	XCZU9EG♠	W8A8	300	283	2147	118	139	36.4	–	73.56⧫	2330.2	–
M ³ ViT [19]	ZCU104♣	INT8	300	–	–	–	–	84	10	0.690*	1217.4	–
ViTCoD [23]	ASIC	W8A8	500	–	–	–	–	–	–	–	–	5.6x
SOLE [120]	ASIC	INT8 + SOLE	–	–	–	–	–	–	–	–	–	57.5x
ME-ViT [134]	U200♣	W8A8	300	288	1024	192	132	94.13	31.8	4.15	–	–

provide customized acceleration, often resulting in lower latency and energy-efficient execution.

VAQF and Auto-ViTAcc apply PTQ optimization to reduce precision while maintaining accuracy, while HeatViT and EQ-ViT optimize workload distribution through kernel-level techniques and adaptive token pruning. On the other hand, MoE introduces irregular memory access patterns, leading to suboptimal hardware utilization. While ping-pong buffering is a well-established technique to mitigate memory stalls, further optimizations are required to exploit hardware parallelism and memory efficiency.

Additionally, Table 15 shows that ASIC designs dominate speedup, but FPGA offers better energy efficiency. This suggests that ASIC accelerators provide raw computational power, while FPGA solutions are more power-efficient but slightly lower in absolute speedup.

Most current acceleration techniques for ViTs are highly customized for specialized hardware, such as FPGAs and ASICs, limiting their deployment flexibility to other platforms. Table 16 highlights the trade-offs between accuracy, adaptability, and hardware compatibility in SW-HW co-design techniques for ViTs. However, limited support for cross-platform compatibility poses challenges for extending these methods to a wide range of edge devices. Among the surveyed techniques, PIVOT [132] emerges as the most flexible solution, as it maintains high accuracy while being compatible with GPPs, unlike ViTCoD, HeatViT, and VAQF, which are hardware-specialized acceleration techniques. Additionally, Table 16 highlights how different acceleration techniques prioritize computations, offering insights into their adaptability across various deployment scenarios.

4.3. Performance analysis for accelerating techniques

This section provides a performance analysis of state-of-the-art (SOTA) accelerating techniques, focusing on key metrics such as power consumption, energy efficiency, resource utilization, and throughput.

4.3.1. Resource utilization

Resource utilization measures the hardware efficiency of various ViT acceleration techniques in terms of the use of block RAM (BRAM), digital signal processing units (DSP), Kilo lookup tables (KLUT), and Kilo flip-flops (KFF) from available resources. As illustrated in Table

17, different acceleration techniques exhibit varying resource consumption patterns, reflecting their optimization strategies and deployment constraints.

From the observation of Table 17, HeatViT (2066 DSPs) [20] and Zhang et al. (2147 DSPs) [133] demonstrate the highest DSP utilization, indicating their reliance on intensive parallel processing to accelerate transformer computations, although both used different FPGA variants. In terms of on-chip memory usage, ViA (1002 BRAMs) [21] and VAQF (565.5 BRAMs) [122] exhibit significant BRAM consumption, emphasizing a design strategy that prioritizes data locality to minimize off-chip memory access latency. In contrast, EQ-ViT (16 BRAMs) [124] utilizes remarkably low BRAM, likely due to its two-level optimization kernels, leveraging both single artificial intelligence engines (AIEs) and AIE array levels. Additionally, Auto-ViT-Acc [123] (186 KLUTs) and HeatViT [20] (161.4 KLUTs) indicate that they need to perform significant logical operations to implement their mixed precision quantization and token pruning to deploy on edge.

4.3.2. Energy efficiency

Table 17 also indicates the energy efficiency of the ViT accelerating techniques. In the current studies, energy efficiency was measured in two ways: using throughput (GOP/J) and using FPS (FPS/W). We also include the power consumption of the accelerating techniques for better accountability. It is perhaps difficult to conclude about energy efficiency when different edge targets are used for the CV task. As illustrated in Table 17, VAQF [122], Auto-ViT-Acc [123], and HeatViT [20] utilized the same FPGA AMD ZCU102 board with the same number of resources and frequency. We can observe that VAQF outperforms Auto-ViT-Acc and HeatViT in terms of energy and power usage. However, we were unable to find any energy comparison from the original paper for ASIC-based accelerating techniques (e.g., ViTCoD, SOLE).

4.3.3. Throughput

Table 17 illustrates the throughput of the accelerating techniques. We observe significant variations in throughput, influenced by factors such as hardware architecture, precision, and optimization strategies. Zhang et al. (2330.2 GOPs) [133] achieve the highest throughput, leveraging an FPGA-based implementation with optimized parallel execution. Auto-ViT-Acc (1181.5 GOPs) and M³ViT (1217.4 GOPs) also

Table 18

Comparison of accuracy across different ViT acceleration techniques using DeiT-B (Base), DeiT-T (Tiny), ViT-B (Base), and ViT-S (Small) as baseline models. Results include Top-1 accuracy on ImageNet-1K and mean Intersection over Union (mIoU) for segmentation benchmarks.

Framework	Baseline	Dataset	Accuracy	
			Top-1 (%)	mIoU
VAQF [122]	DeiT-B	ImageNet-1K [45]	77.6	–
Auto-ViT-Acc [123]	DeiT-B		81.8	–
HeatViT [20]	DeiT-T	ImageNet-1K [45]	72.1	–
EQ-ViT [124]	DeiT-T		74.5	–
ViTCoD [23]	DeiT-T		70.0	–
SOLE [120]	DeiT-T		71.07	–
Zhang et al. [133]	ViT-B	ImageNet-1K [45]	83.1	–
M ³ ViT [19]	ViT-S	PASCAL-Context [110]	–	72.8
		NYUD-v2 [135]	–	45.6

report high throughput, suggesting effective hardware utilization. However, several studies, such as ViTCoD, do not report the throughput. Additionally, EQ-ViT and VAQF balance throughput with power efficiency, offering a more energy-efficient alternative.

4.3.4. Accuracy

Table 18 presents a comparative accuracy analysis across various ViT acceleration techniques on different edge platforms. To ensure a fair and structured comparison, we categorize our evaluation into four widely used ViT-based models: DeiT-Base, DeiT-Tiny, ViT-Base, and ViT-Small. While most acceleration techniques focus on classification tasks using the ImageNet-1K dataset [45].

Among the methods analyzed, Auto-ViT-Acc achieves the highest Top-1 accuracy (81.8%) on DeiT-Base, significantly surpassing VAQF (77.6%) for classification tasks, demonstrating its effectiveness in preserving model accuracy while accelerating inference. For DeiT-Tiny, accuracy varies significantly across different methods: HeatViT (72.1%), EQ-ViT (74.5%), ViTCoD (70.0%), and SOLE (71.07%). Notably, EQ-ViT achieves the highest accuracy among these, highlighting the effectiveness of its attention-based optimizations. However, its energy consumption is significantly higher, indicating a trade-off between accuracy and efficiency.

M³ViT [19] extends its evaluation to segmentation tasks on PASCAL-ContextNYUD-v2 [135]. For segmentation tasks, M³ViT delivers strong performance on PASCAL-Context (72.8 mIoU) but exhibits a noticeable decline on NYUD-v2 (45.6 mIoU), suggesting that its model compression techniques may be dataset-sensitive.

5. Challenges and future directions of ViT on edge devices

ViT models are computation-intensive, and their deployment on resource-constrained edge devices has been a big challenge. However, with the advancement of edge AI, this is now changing, and the efficient and cost-effective implementation of ViT models is possible directly on edge hardware. This increases accessibility for end users and reduces reliance on cloud infrastructure, which lowers latency, improves privacy, and reduces operational costs. However, some areas, such as real-world scenarios and SW-HW co-design still need to be explored for ViT on edge devices. In this section, we will discuss the current challenges and future opportunities of ViT on edge devices.

5.1. Advancement in SW-HW co-design

The SW-HW co-design can reduce the current dilemma between model and hardware architectures. Additionally, different edge hardware platforms (e.g., CPUs, GPUs, and FPGAs) have varying capabilities in handling precision, memory bandwidth, and computational efficiency. Often, accelerators support a uniform bit-width tensor, and

this distinct bit-width precision needs zero padding, incurring inefficient memory usage. It is so hard to optimize the ViT for each type of hardware. Leveraging hardware-aware compression techniques can improve the efficiency of edge deployment. Frameworks such as DNNWeaver [136], VAQF [122], M³ViT [19] have been developed for different hardware platforms like FPGA, GPU accelerators for efficient edge inference. However, most of the current frameworks cannot handle the sparsity caused by model compression. Therefore, the advancement of reconfigurability of SW-HW co-design for handling sparsity can be a future problem to solve.

Additionally, future work should focus on establishing collaborative SW-HW design pipelines that integrate compression-aware modeling with hardware-specific execution constraints. For example, model compression in software could be dynamically adapted based on hardware feedback or user-specific hardware requirements. This requires tighter integration between models and hardware profiling tools. Further, hardware-agnostic intermediate representations and unified compiler toolchains could help bridge the gap between diverse ViT models and heterogeneous edge devices.

5.2. Utilizing NAS for inference and compression parameter search

NAS is widely used to discover efficient model structures. However, its application to ViTs on edge devices remains limited due to high computational costs and strong coupling to specific hardware platforms. Frameworks such as HAQ [85] and APQ [137] utilized NAS to automate pruning and quantization strategies through reinforcement learning or evolutionary algorithms. However, these works are highly customized for specific hardware (e.g., HAQ for FPGA). ProxylessNAS [138] is a notable effort in this direction that introduces hardware-aware search and demonstrates strong real-time latency prediction, but it is limited to CNN architectures. Extending such approaches to ViT and its hierarchical, attention-based structure remains an open challenge.

Future research should extend NAS to simultaneously search for model-independent compression parameters, such as quantization bit-widths, pruning ratios, and distillation strategies, based on hardware-aware objectives (e.g., latency, memory usage, energy efficiency). Reinforcement learning can be utilized to efficiently guide the search process based on hardware requirements. Additionally, exploring NAS to FPGA custom design, which requires deep hardware expertise, might be an interesting and challenging research area for the efficient deployment of ViT.

5.3. Hardware accelerators to handle sparsity and mixed precision

Traditional processors such as GPUs, CPUs, or even FPGAs struggle with executing irregular tensor operations efficiently. Current accelerators are not inherently designed to process sparse tensors, as they

often fetch zero values from memory to processing elements (PEs), incurring unnecessary latency and energy overhead. Another challenge is the limited support for mixed-precision execution. Although recent techniques have shown that mixed-precision quantization can preserve ViT accuracy while reducing model size, most edge hardware requires uniform bit-widths across operations [139], leading to compatibility issues or bit padding and making it hard to deploy on edge devices.

To address these limitations, future research should focus on developing sparsity-aware accelerator architectures capable of dynamically skipping zero computations, using methods such as compressed sparse row (CSR) encoding, zero-skipping logic [140], or sparsity-driven scheduling techniques [141,142]. Similarly, designing bit-level reconfigurable compute units could enable native support for mixed-precision execution without introducing significant hardware redundancy. Thus, specialized techniques are needed to optimize the storage and computation of nonzero values in ViT.

Additionally, emerging math-based accelerating techniques such as tensor-train (TT) decomposition offer a promising direction for accelerating ViT models. TT-decomposition can significantly reduce parameter count and computational complexity by representing high-dimensional weight matrices in a compact, low-rank format. Recent studies, including TT-ViT [143] and TT-MLP [144], demonstrate its potential for transformer and MLP compression, while hardware-aware implementations [145] further highlight its relevance for deployment. Although integration into edge-specific ViT deployments remains limited, TT-decomposition represents a promising future research avenue for ViT acceleration, especially on FPGA due to its reconfigurable property.

5.4. Automated edge aware model compression

Most of the current model compression techniques require manual adjustment of hyperparameters such as quantization bit width, pruning ratio, or layer-wise sparsity. Compression hyperparameters must be adjusted automatically or adaptively within the resource budget with minimum degradation of accuracy. From our observation, few works explore adjusting the compression parameters automatically. Sparse-ViT [146] is one of the works that effectively reduced computation by targeting less-important regions with dynamically chosen pruning ratios in the images for the ViT model, achieving significant latency reductions. As a result, developing a hardware-efficient automaticity compression technique can be an interesting research domain in the future.

Another drawback we observe from Section 2.3.2 is that most of the current work on ViT is post-training. The most interesting reason, perhaps the interactive nature of the training process, is that implementing compression techniques during training requires cost and time. However, QAT techniques on ViT are promising [72,104,105,112], but there is limited work on other compression techniques. Thus, the automated exploration of compression techniques during training can improve hardware realization. However, efficient compression techniques for faster convergence during training with reduced computation need to be explored in the future.

5.5. Developing benchmarks

Proper benchmark standards to evaluate the performance of the edge devices are essential. The different stages of the model for deploying edge, including compression and accelerators, require a universal and comprehensive set of metrics to compare different proposed solutions. However, benchmarking datasets and models from the system perspective is limited. For example, most of the compression techniques for ViT were evaluated on the ImageNet-1k [45] dataset for classification tasks and the COCO-2017 [103] datasets for object detection tasks. However, expanding that knowledge to different real-world application areas is still limited due to the lack of datasets and model benchmarks. Thus, more benchmarking datasets and ViT models for evaluating

the proposed system/framework need to be developed for different application areas, such as medical imaging and autonomous driving. In addition, making one compression technique universal for different CV tasks is challenging. Making task-independent universal compression techniques can be an interesting research domain in the future.

5.6. Real-world case studies

The deployment of ViT on edge devices has gained significant progress in recent years. However, most of the compression techniques, frameworks, and accelerators are limited to evaluation in an academic environment. For instance, there are numerous ViT models on medical imaging datasets for different CV tasks such as image classification [147–149], segmentation [10,150–153], and object detection [154–157]. However, few studies have evaluated compression and accelerator techniques on those ViT-based medical imaging models. Exploring ViT on edge for medical imaging can be challenging because of the unique nature of the data (3D ultrasounds or MRIs).

Maintaining accuracy, latency, and precision is critical in real-world applications, particularly in critical fields like medical imaging. In medical imaging, a significant challenge during compression of the ViT models is maintaining the spatial resolution and feature details since even small degradations in accuracy will affect the diagnostic outcome significantly [158]. Compression techniques can reduce excessive feature abstraction, potentially discarding vital low-level details essential for accurate diagnoses. Furthermore, transfer learning in medical imaging adds another layer of complexity—determining which pre-trained layers to retain or modify without losing critical learned representations is a significant challenge [159,160]. Therefore, it is an open research direction to achieve a balance between model efficiency and diagnostic reliability for ViT models on edge for real-world scenarios such as healthcare applications.

5.7. Seamless model-to-edge integration

The conversion from a trained model into a hardware-compatible version for inference requires extensive time, cost, and, most importantly, manual labor in each step. Additionally, there is a high knowledge gap between the research community. For example, training new models requires extensive software knowledge, while compressing and developing accelerating strategies require deep hardware knowledge. It is challenging to find an expert in both directions in the research community. These difficulties create a significant research gap in developing tools that automatically map the models to hardware. However, FPGA can overcome some limitations with the ability to reconfigure new operations and modules. However, the available tools are insufficient for the automatic mapping of models and are more limited for ViT.

The current deep learning frameworks for edge deployment help researchers quickly prototype the models to deploy on edge. However, it lacks support with the rapid growth of different model architectures. Additionally, most of the current frameworks are evaluated for CNN models, while those frameworks are still in the experimental phase for ViT. For instance, Xilinx provides quantization support through the FINN-R framework for inference realization on FPGA [161], limiting only standard techniques for ViT. Therefore, the automatic mapping from model to edge, precisely a one-click solution for deployment on

edge based on the budget or auto-generated compression techniques, can be an interesting domain in the future.

5.8. Robustness to diverse data modality

In real-world scenarios, data sources change from sensor to sensor or vendor to vendor. For example, medical imaging includes X-rays and other modalities like magnetic resonance imaging (MRI), computed tomography (CT) scans, and ultrasounds. Each modality has its characteristics, and a compression technique effective for one might not be for another. So, using a generic model compression technique for all modalities is always tricky. Such heterogeneity may cause inconsistency in data, imposing a challenge on edge performance. Federated learning, multi-modal fusion, and adaptive data calibration techniques can be promising research directions to mitigate the data inconsistency problem at the edge device.

6. Conclusion

In this survey, we comprehensively analyze model compression and hardware acceleration techniques on ViT models, spanning techniques ranging from pruning, quantization, and KD, software tools, to SW-HW co-design. Our analysis indicates that while SOTA ViT compression and acceleration techniques effectively reduce computational overhead and improve inference speed, challenges such as hardware adaptability, memory bottlenecks, and optimal compression strategies remain unexplored. Additionally, we discuss the potential future directions, such as utilizing NAS to find hardware-aware optimization parameters, sparsity-aware accelerators, and efficient cross-platform SW-HW co-design frameworks.

Overall, optimizing ViT for edge devices is an evolving field, marked by both challenges and opportunities. This survey provides a comprehensive overview of the current landscape and aims to inspire continued innovation in this crucial area.

CRediT authorship contribution statement

Shaibal Saha: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Lanyu Xu:** Writing – review & editing, Supervision, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Lanyu Xu reports financial support was provided by National Science Foundation. Lanyu Xu reports financial support was provided by Michigan Space Grant Consortium. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partly supported by the U.S. National Science Foundation under Grants CNS-2245729 and Michigan Space Grant Consortium, United States 80NSSC20M0124. The authors would also like to thank Yunge Li for the continuous discussions and insightful feedback that helped shape the direction of this study.

Data availability

No data was used for the research described in the article.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017).
- [2] S. Hochreiter, Long short-term memory, *Neural Comput.* MIT- Press (1997).
- [3] J. Chung, C. Gulcehre, K. Cho, Y. Bengio, Empirical evaluation of gated recurrent neural networks on sequence modeling, 2014, arXiv preprint arXiv:1412.3555.
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., An image is worth 16x16 words: Transformers for image recognition at scale, 2020, arXiv preprint arXiv:2010.11929.
- [5] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Adv. Neural Inf. Process. Syst.* 25 (2012).
- [6] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, B. Guo, Swin transformer: Hierarchical vision transformer using shifted windows, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10012–10022.
- [7] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, End-to-end object detection with transformers, in: *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part I* 16, Springer, 2020, pp. 213–229.
- [8] Z. Zhang, X. Lu, G. Cao, Y. Yang, L. Jiao, F. Liu, ViT-YOLO: Transformer-based YOLO for object detection, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 2799–2808.
- [9] A. Hatamizadeh, Y. Tang, V. Nath, D. Yang, A. Myronenko, B. Landman, H.R. Roth, D. Xu, Unetr: Transformers for 3d medical image segmentation, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 574–584.
- [10] Z. Li, Y. Li, Q. Li, P. Wang, D. Guo, L. Lu, D. Jin, Y. Zhang, Q. Hong, Lvit: language meets vision transformer in medical image segmentation, *IEEE Trans. Med. Imaging* (2023).
- [11] M. Dehghani, J. Djolonga, B. Mustafa, P. Padlewski, J. Heek, J. Gilmer, A.P. Steiner, M. Caron, R. Geirhos, I. Alabdulmohsin, et al., Scaling vision transformers to 22 billion parameters, in: *International Conference on Machine Learning*, PMLR, 2023, pp. 7480–7512.
- [12] T. Liang, J. Glossner, L. Wang, S. Shi, X. Zhang, Pruning and quantization for deep neural network acceleration: A survey, *Neurocomputing* 461 (2021) 370–403.
- [13] H. Bai, H. Mao, D. Nair, Dynamically pruning segformer for efficient semantic segmentation, in: *ICASSP 2022–2022 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP, IEEE*, 2022, pp. 3298–3302.
- [14] Y. Lin, T. Zhang, P. Sun, Z. Li, S. Zhou, FQ-vit: Post-training quantization for fully quantized vision transformer, in: *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, 2022, pp. 1173–1179.
- [15] Z. Li, J. Xiao, L. Yang, Q. Gu, Repq-vit: Scale reparameterization for post-training quantization of vision transformers, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17227–17236.
- [16] J. Gou, B. Yu, S.J. Maybank, D. Tao, Knowledge distillation: A survey, *Int. J. Comput. Vis.* 129 (2021) 1789–1819.
- [17] S. Lin, H. Xie, B. Wang, K. Yu, X. Chang, X. Liang, G. Wang, Knowledge distillation via the target-aware transformer, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10915–10924.
- [18] Dimensions.ai, Dimensions analytics: Research publication trends, 2025, URL <https://app.dimensions.ai/> (Accessed 23 February 2025).
- [19] Z. Fan, R. Sarkar, Z. Jiang, T. Chen, K. Zou, Y. Cheng, C. Hao, Z. Wang, et al., M³vit: Mixture-of-experts vision transformer for efficient multi-task learning with model-accelerator co-design, *Adv. Neural Inf. Process. Syst.* 35 (2022) 28441–28457.
- [20] P. Dong, M. Sun, A. Lu, Y. Xie, K. Liu, Z. Kong, X. Meng, Z. Li, X. Lin, Z. Fang, et al., Heatvit: Hardware-efficient adaptive token pruning for vision transformers, in: *2023 IEEE International Symposium on High-Performance Computer Architecture, HPCA, IEEE*, 2023, pp. 442–455.
- [21] T. Wang, L. Gong, C. Wang, Y. Yang, Y. Gao, X. Zhou, H. Chen, Via: A novel vision-transformer accelerator based on fpga, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 41 (11) (2022) 4088–4099.
- [22] S. Nag, G. Datta, S. Kundu, N. Chandrachoodan, P.A. Beerel, Vita: A vision transformer inference accelerator for edge applications, in: *2023 IEEE International Symposium on Circuits and Systems, ISCAS*, 2023, pp. 1–5, <http://dx.doi.org/10.1109/ISCAS46773.2023.10181988>.
- [23] H. You, Z. Sun, H. Shi, Z. Yu, Y. Zhao, Y. Zhang, C. Li, B. Li, Y. Lin, Vitcod: Vision transformer acceleration via dedicated algorithm and accelerator co-design, in: *2023 IEEE International Symposium on High-Performance Computer Architecture, HPCA, IEEE*, 2023, pp. 273–286.
- [24] M. Chen, J. Gao, W. Yu, Lightweight and optimization acceleration methods for vision transformer: A review, in: *2022 IEEE 25th International Conference on Intelligent Transportation Systems, ITSC, IEEE*, 2022, pp. 2154–2160.

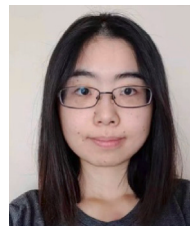
- [25] S. Huang, E. Tang, S. Li, X. Ping, R. Chen, Hardware-friendly compression and hardware acceleration for transformer: A survey., *Electron. Res. Arch.* 30 (10) (2022).
- [26] L. Papa, P. Russo, I. Amerini, L. Zhou, A survey on efficient vision transformers: algorithms, techniques, and performance benchmarking, *IEEE Trans. Pattern Anal. Mach. Intell.* (2024).
- [27] D. Du, G. Gong, X. Chu, Model quantization and hardware acceleration for vision transformers: A comprehensive survey, 2024, arXiv preprint arXiv:2405.00314.
- [28] S. Hanson, L. Pratt, Comparing biases for minimal network construction with back-propagation, *Adv. Neural Inf. Process. Syst.* 1 (1988).
- [29] B. Hassibi, D. Stork, Second order derivatives for network pruning: Optimal brain surgeon, *Adv. Neural Inf. Process. Syst.* 5 (1992).
- [30] P. Molchanov, S. Tyree, T. Karras, T. Aila, J. Kautz, Pruning convolutional neural networks for resource efficient inference, in: *International Conference on Learning Representations*, 2017.
- [31] Y. Cheng, D. Wang, P. Zhou, T. Zhang, A survey of model compression and acceleration for deep neural networks, 2017, arXiv preprint arXiv:1710.09282.
- [32] T. Wu, J. Shi, D. Zhou, X. Zheng, N. Li, Evolutionary multi-objective one-shot filter pruning for designing lightweight convolutional neural network, *Sensors* 21 (17) (2021) 5901.
- [33] L. Cai, J. Wang, L. Yu, B. Yan, Y. Tao, Y. Yang, Accelerating neural-ODE inference on FPGAs with two-stage structured pruning and history-based step-size search, in: *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays, FPGA '23*, Association for Computing Machinery, New York, NY, USA, 2023, pp. 177–183, <http://dx.doi.org/10.1145/3543622.3573044>.
- [34] Y. He, L. Xiao, Structured pruning for deep convolutional neural networks: A survey, 2023, arXiv preprint arXiv:2303.00566.
- [35] S. Han, J. Pool, J. Tran, W. Dally, Learning both weights and connections for efficient neural network, *Adv. Neural Inf. Process. Syst.* 28 (2015).
- [36] Z. Song, Y. Xu, Z. He, L. Jiang, N. Jing, X. Liang, Cp-vit: Cascade vision transformer pruning via progressive sparsity prediction, 2022, arXiv preprint arXiv:2203.04570.
- [37] Z. Kong, P. Dong, X. Ma, X. Meng, W. Niu, M. Sun, X. Shen, G. Yuan, B. Ren, H. Tang, et al., Spvit: Enabling faster vision transformers via latency-aware soft token pruning, in: *Computer Vision—ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XI*, Springer, 2022, pp. 620–640.
- [38] M. Zhu, Y. Tang, K. Han, Vision transformer pruning, 2021, arXiv preprint arXiv:2104.08500.
- [39] Y. Tang, K. Han, Y. Wang, C. Xu, J. Guo, C. Xu, D. Tao, Patch slimming for efficient vision transformers, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12165–12174.
- [40] F. Yu, K. Huang, M. Wang, Y. Cheng, W. Chu, L. Cui, Width & depth pruning for vision transformers, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 36, 2022, pp. 3143–3151.
- [41] S. Yu, T. Chen, J. Shen, H. Yuan, J. Tan, S. Yang, J. Liu, Z. Wang, Unified visual transformer compression, in: *International Conference on Learning Representations*, 2022, URL <https://openreview.net/forum?id=9jsZiUgkCZP>.
- [42] H. Yang, H. Yin, M. Shen, P. Molchanov, H. Li, J. Kautz, Global vision transformer pruning with hessian-aware saliency, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 18547–18557.
- [43] T. Chen, Y. Cheng, Z. Gan, L. Yuan, L. Zhang, Z. Wang, Chasing sparsity in vision transformers: An end-to-end exploration, *Adv. Neural Inf. Process. Syst.* 34 (2021) 19974–19988.
- [44] Z. Wang, H. Luo, P. Wang, F. Ding, F. Wang, H. Li, Vtc-lfc: Vision transformer compression with low-frequency components, *Adv. Neural Inf. Process. Syst.* 35 (2022) 13974–13988.
- [45] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, ImageNet: A large-scale hierarchical image database, in: 2009 IEEE Conference on Computer Vision and Pattern Recognition, 2009, pp. 248–255, <http://dx.doi.org/10.1109/CVPR.2009.5206848>.
- [46] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, H. Jégou, Training data-efficient image transformers & distillation through attention, in: *International Conference on Machine Learning, PMLR*, 2021, pp. 10347–10357.
- [47] C. Zheng, K. Zhang, Z. Yang, W. Tan, J. Xiao, Y. Ren, S. Pu, et al., Savit: Structure-aware vision transformer pruning via collaborative optimization, *Adv. Neural Inf. Process. Syst.* 35 (2022) 9010–9023.
- [48] N. Zheng, H. Jiang, Q. Zhang, Z. Han, L. Ma, Y. Yang, F. Yang, C. Zhang, L. Qiu, M. Yang, et al., Pit: Optimization of dynamic sparse deep learning models via permutation invariant transformation, in: *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023, pp. 331–347.
- [49] S. Sah, R. Kumar, H. Rohmetra, E. Saboori, Token pruning using a lightweight background aware vision transformer, in: *NeurIPS 2024 Workshop on Fine-Tuning in Modern Machine Learning: Principles and Scalability*, 2024, URL <https://openreview.net/forum?id=6ZaSMpmNVF>.
- [50] Q. Tang, B. Zhang, J. Liu, F. Liu, Y. Liu, Dynamic token pruning in plain vision transformers for semantic segmentation, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 777–786.
- [51] Z. Hou, S.-Y. Kung, Multi-dimensional vision transformer compression via dependency guided gaussian process search, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 3669–3678.
- [52] A. Krizhevsky, V. Nair, G. Hinton, Cifar-10 (canadian institute for advanced research) <http://www.cs.toronto.edu/kriz/cifar.html> 5 (4) (2010) 1..
- [53] K. Xu, Z. Wang, C. Chen, X. Geng, J. Lin, X. Yang, M. Wu, X. Li, W. Lin, LPViT: Low-power semi-structured pruning for vision transformers, in: A.S. Leonardis, E. Ricci, S. Roth, O. Russakovsky, T. Sattler, G. Varol (Eds.), *Computer Vision – ECCV 2024*, Springer Nature Switzerland, Cham, 2025, pp. 269–287.
- [54] Z. Li, S. Ghodrati, A. Yazdanbakhsh, H. Esmailzadeh, M. Kang, Accelerating attention through gradient-based learned runtime pruning, in: *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 902–915.
- [55] K. Wu, J. Zhang, H. Peng, M. Liu, B. Xiao, J. Fu, L. Yuan, Tinyvit: Fast pretraining distillation for small vision transformers, in: *European Conference on Computer Vision*, Springer, 2022, pp. 68–85.
- [56] X. Chen, Q. Cao, Y. Zhong, J. Zhang, S. Gao, D. Tao, Dearth: data-efficient early knowledge distillation for vision transformers, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12052–12062.
- [57] S. Ren, Z. Gao, T. Hua, Z. Xue, Y. Tian, S. He, H. Zhao, Co-advise: Cross inductive bias distillation, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 16773–16782.
- [58] Z. Hao, J. Guo, D. Jia, K. Han, Y. Tang, C. Zhang, H. Hu, Y. Wang, Learning efficient vision transformers via fine-grained manifold distillation, *Adv. Neural Inf. Process. Syst.* 35 (2022) 9164–9175.
- [59] J. Zhang, H. Peng, K. Wu, M. Liu, B. Xiao, J. Fu, L. Yuan, Minivit: Compressing vision transformers with weight multiplexing, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12145–12154.
- [60] J. Wang, M. Cao, S. Shi, B. Wu, Y. Yang, Attention probe: Vision transformer distillation in the wild, in: *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP*, 2022, pp. 2220–2224, <http://dx.doi.org/10.1109/ICASSP43922.2022.9747484>.
- [61] C. Yang, X. Yu, Z. An, Y. Xu, Categories of response-based, feature-based, and relation-based knowledge distillation, in: *Advancements in Knowledge Distillation: Towards New Horizons of Intelligent Systems*, Springer, 2023, pp. 1–32.
- [62] L. Deng, The mnist database of handwritten digit images for machine learning research, *IEEE Signal Process. Mag.* 29 (6) (2012) 141–142.
- [63] B. Zhou, H. Zhao, X. Puig, T. Xiao, S. Fidler, A. Barriuso, A. Torralba, Semantic understanding of scenes through the ade20k dataset, *Int. J. Comput. Vis.* 127 (3) (2019) 302–321.
- [64] H. Caesar, J. Uijlings, V. Ferrari, COCO-stuff: Thing and stuff classes in context, in: *Computer Vision and Pattern Recognition (CVPR)*, 2018 IEEE Conference on, IEEE, 2018.
- [65] M. Everingham, L. Gool, C.K. Williams, J. Winn, A. Zisserman, The pascal visual object classes (VOC) challenge, *Int. J. Comput. Vis.* 88 (2) (2010) 303–338, <http://dx.doi.org/10.1007/s11263-009-0275-4>.
- [66] Z. Yuan, C. Xue, Y. Chen, Q. Wu, G. Sun, Pqt4vit: Post-training quantization for vision transformers with twin uniform quantization, in: *European Conference on Computer Vision*, Springer, 2022, pp. 191–207.
- [67] J. Fang, A. Shafiee, H. Abdel-Aziz, D. Thorsley, G. Georgiadis, J.H. Hassoun, Post-training piecewise linear quantization for deep neural networks, in: *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part II* 16, Springer, 2020, pp. 69–86.
- [68] S. Oh, H. Sim, J. Kim, J. Lee, Non-uniform step size quantization for accurate post-training quantization, in: *European Conference on Computer Vision*, Springer, 2022, pp. 658–673.
- [69] Y. Jeon, C. Lee, E. Cho, Y. Ro, Mr. biq: Post-training non-uniform quantization based on minimizing the reconstruction error, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12329–12338.
- [70] Z. Liu, Y. Wang, K. Han, W. Zhang, S. Ma, W. Gao, Post-training quantization for vision transformer, *Adv. Neural Inf. Process. Syst.* 34 (2021) 28092–28103.
- [71] Z. Wang, C. Wang, X. Xu, J. Zhou, J. Lu, Quantformer: Learning extremely low-precision vision transformers, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (7) (2023) 8813–8826, <http://dx.doi.org/10.1109/TPAMI.2022.3229313>.
- [72] Y. Zhang, Z. Dong, H. Yang, M. Lu, C.-C. Tseng, Y. Du, K. Keutzer, L. Du, S. Zhang, QD-BEV: Quantization-aware view-guided distillation for multi-view 3D object detection, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 3825–3835.
- [73] Y. Li, S. Xu, B. Zhang, X. Cao, P. Gao, G. Guo, Q-vit: Accurate and fully quantized low-bit vision transformer, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), *Advances in Neural Information Processing Systems*, 35, Curran Associates, Inc., 2022, pp. 34451–34463, URL https://proceedings.neurips.cc/paper_files/paper/2022/file/deb921bff461a7b0a5c344a4871e7101-Paper-Conference.pdf.
- [74] H. Fan, G. Wang, M. Ferianc, X. Niu, W. Luk, Static block floating-point quantization for convolutional neural networks on FPGA, in: 2019 International Conference on Field-Programmable Technology, ICFPT, IEEE, 2019, pp. 28–35.

- [75] F. Liu, W. Zhao, Z. He, Y. Wang, Z. Wang, C. Dai, X. Liang, L. Jiang, Improving neural network efficiency via post-training quantization with adaptive floating-point, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 5281–5290.
- [76] Z. Liu, Y. Wang, K. Han, S. Ma, W. Gao, Instance-aware dynamic neural network quantization, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12434–12443.
- [77] K. Huang, B. Li, D. Xiong, H. Jiang, X. Jiang, X. Yan, L. Claesen, D. Liu, J. Chen, Z. Liu, Structured dynamic precision for deep neural networks quantization, *ACM Trans. Des. Autom. Electron. Syst.* 28 (1) (2023) 1–24.
- [78] R. Li, Y. Wang, F. Liang, H. Qin, J. Yan, R. Fan, Fully quantized network for object detection, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2810–2819.
- [79] T. Chu, Q. Luo, J. Yang, X. Huang, Mixed-precision quantized neural networks with progressively decreasing bitwidth, *Pattern Recognit.* 111 (2021) 107647, <http://dx.doi.org/10.1016/j.patcog.2020.107647>, URL <https://www.sciencedirect.com/science/article/pii/S0031320320304507>.
- [80] N. Ranjan, A. Savakis, LRP-QViT: Mixed-precision vision transformer quantization via layer-wise relevance propagation, 2024, arXiv preprint arXiv:2401.11243.
- [81] F. Zhang, N. Liu, Y. Hu, F. Duan, Mffnet: Single facial depth map refinement using multi-level feature fusion, *Signal Process., Image Commun.* 103 (2022) 116649.
- [82] Y. Xie, X. Hou, Y. Guo, X. Wang, J. Zheng, Joint-guided distillation binary neural network via dynamic channel-wise diversity enhancement for object detection, *IEEE Trans. Circuits Syst. Video Technol.* (2023).
- [83] T. Chu, Q. Luo, J. Yang, X. Huang, Mixed-precision quantized neural networks with progressively decreasing bitwidth, *Pattern Recognit.* 111 (2021) 107647.
- [84] S. Xu, Y. Li, M. Lin, P. Gao, G. Guo, J. Lü, B. Zhang, Q-DETR: An efficient low-bit quantized detection transformer, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 3842–3851.
- [85] K. Wang, Z. Liu, Y. Lin, J. Lin, S. Han, Haq: Hardware-aware automated quantization with mixed precision, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 8612–8620.
- [86] Z. Dong, Y. Gao, Q. Huang, J. Wawrzyniak, H.K. So, K. Keutzer, Hao: Hardware-aware neural architecture optimization for efficient inference, in: *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines, FCCM*, IEEE, 2021, pp. 50–59.
- [87] M. Nagel, M. Fournarakis, R.A. Amjad, Y. Bondarenko, M. Van Baalen, T. Blankevoort, A white paper on neural network quantization, 2021, arXiv preprint arXiv:2106.08295.
- [88] Y. Ding, H. Qin, Q. Yan, Z. Chai, J. Liu, X. Wei, X. Liu, Towards accurate post-training quantization for vision transformer, in: *Proceedings of the 30th ACM International Conference on Multimedia*, 2022, pp. 5380–5388.
- [89] L. Niu, J. Liu, Z. Yuan, D. Yang, X. Wang, W. Liu, Improving post-training quantization on object detection with task loss-guided lp metric, 2023, arXiv preprint arXiv:2304.09785.
- [90] D. Zheng, Y. Liu, L. Li, et al., Leveraging inter-layer dependency for post-training quantization, *Adv. Neural Inf. Process. Syst.* 35 (2022) 6666–6679.
- [91] C. Lv, H. Chen, J. Guo, Y. Ding, X. Liu, PTQ4sam: Post-training quantization for segment anything, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 15941–15951.
- [92] Y. Liu, H. Yang, Z. Dong, K. Keutzer, L. Du, S. Zhang, NoisyQuant: Noisy bias-enhanced post-training activation quantization for vision transformers, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 20321–20330.
- [93] Z. Li, Q. Gu, I-vit: Integer-only quantization for efficient vision transformer inference, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17065–17075.
- [94] S. Kim, A. Gholami, Z. Yao, M.W. Mahoney, K. Keutzer, I-bert: Integer-only bert quantization, in: *International Conference on Machine Learning*, PMLR, 2021, pp. 5506–5518.
- [95] Y.-S. Tai, et al., Mptq-vit: Mixed-precision post-training quantization for vision transformer, 2024, arXiv preprint arXiv:2401.14895.
- [96] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han, Smoothquant: Accurate and efficient post-training quantization for large language models, in: *International Conference on Machine Learning*, PMLR, 2023, pp. 38087–38099.
- [97] Y.-S. Tai, M.-G. Lin, A.-Y.A. Wu, TSPTQ-vit: Two-scaled post-training quantization for vision transformer, in: *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP*, 2023, pp. 1–5, <http://dx.doi.org/10.1109/ICASSP49357.2023.10096817>.
- [98] Y. Zhong, J. Hu, Y. Huang, Y. Zhang, R. Ji, ERQ: Error reduction for post-training quantization of vision transformers, in: *Forty-First International Conference on Machine Learning*, 2024.
- [99] M. Nagel, R.A. Amjad, M. Van Baalen, C. Louizos, T. Blankevoort, Up or down? adaptive rounding for post-training quantization, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 7197–7206.
- [100] Z. Wu, J. Chen, H. Zhong, D. Huang, Y. Wang, AdaLog: Post-training quantization for vision transformers with adaptive logarithm quantizer, in: *European Conference on Computer Vision*, Springer, 2025, pp. 411–427.
- [101] A. Ramachandran, S. Kundu, T. Krishna, CLAMP-vit: contrastive data-free learning for adaptive post-training quantization of vits, in: *European Conference on Computer Vision*, Springer, 2025, pp. 307–325.
- [102] S. Wu, G. Li, L. Deng, L. Liu, D. Wu, Y. Xie, L. Shi, L1-Norm batch normalization for efficient training of deep neural networks, *IEEE Trans. Neural Networks Learn. Syst.* 30 (7) (2018) 2043–2051.
- [103] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C.L. Zitnick, P. Dollár, Microsoft COCO: Common objects in context, 2015, arXiv:1405.0312.
- [104] R. Wang, H. Sun, L. Yang, S. Lin, C. Liu, Y. Gao, Y. Hu, B. Zhang, AQ-DETR: Low-bit quantized detection transformer with auxiliary queries, *Proc. the AAAI Conf. Artif. Intell.* 38 (14) (2024) 15598–15606, <http://dx.doi.org/10.1609/aaai.v38i14.29487>, URL <https://ojs.aaai.org/index.php/AAAI/article/view/29487>.
- [105] Z. Li, M. Chen, J. Xiao, Q. Gu, PSAQ-vit v2: Toward accurate and general data-free quantization for vision transformers, *IEEE Trans. Neural Networks Learn. Syst.* (2023).
- [106] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C.L. Zitnick, Microsoft coco: Common objects in context, in: *Computer Vision—ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V 13*, Springer, 2014, pp. 740–755.
- [107] D. Wu, Q. Tang, Y. Zhao, M. Zhang, Y. Fu, D. Zhang, Easyquant: Post-training quantization via scale optimization, 2020, arXiv preprint arXiv:2006.16669.
- [108] Y. Choukroun, E. Kravchik, F. Yang, P. Kisilev, Low-bit quantization of neural networks for efficient inference, in: *2019 IEEE/CVF International Conference on Computer Vision Workshop, ICCVW*, IEEE, 2019, pp. 3009–3018.
- [109] S.K. Esser, J.L. McKinstry, D. Bablani, R. Appuswamy, D.S. Modha, LEARNED STEP size quantization, in: *International Conference on Learning Representations*, 2020, URL <https://openreview.net/forum?id=rkgO66VKDS>.
- [110] M. Everingham, L. Van Gool, C.K. Williams, J. Winn, A. Zisserman, The pascal visual object classes (voc) challenge, *Int. J. Comput. Vis.* 88 (2010) 303–338.
- [111] Z. Li, L. Ma, M. Chen, J. Xiao, Q. Gu, Patch similarity aware data-free quantization for vision transformers, in: *European Conference on Computer Vision*, 2022, pp. 154–170.
- [112] P. Dong, L. Lu, C. Wu, C. Lyu, G. Yuan, H. Tang, Y. Wang, Packqv: Faster sub-8-bit vision transformers via full and packed quantization on the mobile, *Adv. Neural Inf. Process. Syst.* 36 (2024).
- [113] H. Caesar, V. Bankiti, A.H. Lang, S. Vora, V.E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, O. Beijbom, Nuscenes: A multimodal dataset for autonomous driving, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 11621–11631.
- [114] M. Nagel, M.v. Baalen, T. Blankevoort, M. Welling, Data-free quantization through weight equalization and bias correction, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1325–1334.
- [115] Z. Yao, Z. Dong, Z. Zheng, A. Gholami, J. Yu, E. Tan, L. Wang, Q. Huang, Y. Wang, M. Mahoney, et al., Hawq-v3: Dyadic neural network quantization, in: *International Conference on Machine Learning*, PMLR, 2021, pp. 11875–11886.
- [116] J. Choi, Z. Wang, S. Venkataramani, P.I.-J. Chuang, V. Srinivasan, K. Gopalakrishnan, PACT: Parameterized clipping activation for quantized neural networks, 2018, URL <https://openreview.net/forum?id=BySugjCb>.
- [117] Z. Li, W. Wang, H. Li, E. Xie, C. Sima, T. Lu, Y. Qiao, J. Dai, Bevformer: Learning bird’s-eye-view representation from multi-camera images via spatiotemporal transformers, in: *European Conference on Computer Vision*, Springer, 2022, pp. 1–18.
- [118] J.R. Stevens, R. Venkatesan, S. Dai, B. Khailany, A. Raghunathan, Softmax: Hardware/software co-design of an efficient softmax for transformers, in: *2021 58th ACM/IEEE Design Automation Conference, DAC*, IEEE, 2021, pp. 469–474.
- [119] Z. Zhang, B. He, Z. Zhang, Practical edge kernels for integer-only vision transformers under post-training quantization, *Proc. Mach. Learn. Syst.* 5 (2023) 35–47.
- [120] W. Wang, S. Zhou, W. Sun, P. Sun, Y. Liu, SOLE: Hardware-software co-design of softmax and LayerNorm for efficient transformer inference, in: *2023 IEEE/ACM International Conference on Computer Aided Design, ICCAD*, IEEE, 2023, pp. 1–9.
- [121] A. Marchisio, D. Dura, M. Capra, M. Martina, G. Masera, M. Shafique, SwiftTron: An efficient hardware accelerator for quantized transformers, in: *2023 International Joint Conference on Neural Networks, IJCNN*, IEEE, 2023, pp. 1–9.
- [122] M. Sun, H. Ma, G. Kang, Y. Jiang, T. Chen, X. Ma, Z. Wang, Y. Wang, Vaqf: Fully automatic software-hardware co-design framework for low-bit vision transformer, 2022, arXiv preprint arXiv:2201.06618.
- [123] Z. Lit, M. Sun, A. Lu, H. Ma, G. Yuan, Y. Xie, H. Tang, Y. Li, M. Leeser, Z. Wang, et al., Auto-vit-acc: An fpga-aware automatic acceleration framework for vision transformer with mixed-scheme quantization, in: *2022 32nd International Conference on Field-Programmable Logic and Applications, FPL*, IEEE, 2022, pp. 109–116.
- [124] P. Dong, J. Zhuang, Z. Yang, S. Ji, Y. Li, D. Xu, H. Huang, J. Hu, A.K. Jones, Y. Shi, Y. Wang, P. Zhou, EQ-vit: Algorithm-hardware co-design for end-to-end acceleration of real-time vision transformer inference on versal ACAP architecture, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 43 (11) (2024) 3949–3960, <http://dx.doi.org/10.1109/TCAD.2024.3443692>.

- [125] X. Liu, Y. Song, X. Li, Y. Sun, H. Lan, Z. Liu, L. Jiang, J. Li, ED-vit: Splitting vision transformer for distributed inference on edge devices, 2024, arXiv preprint [arXiv:2410.11650](https://arxiv.org/abs/2410.11650).
- [126] Z. Wang, G. Wang, G. He, COSA plus: Enhanced co-operative systolic arrays for attention mechanism in transformers, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 44 (2) (2025) 723–736, <http://dx.doi.org/10.1109/TCAD.2024.3434447>.
- [127] F. Akopyan, J. Sawada, A. Cassidy, R. Alvarez-Icaza, J. Arthur, P. Merolla, N. Imam, Y. Nakamura, P. Datta, G.-J. Nam, et al., Truenorth: Design and tool flow of a 65 mw 1 million neuron programmable neurosynaptic chip, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 34 (10) (2015) 1537–1557.
- [128] N. Corporation, NVIDIA deep learning accelerator (NVDLA), 2021, URL <http://nvidia.org> (Accessed 07 April 2025).
- [129] L. Cavigelli, L. Benini, Origami: A 803-GOp/s/W convolutional network accelerator, *IEEE Trans. Circuits Syst. Video Technol.* 27 (11) (2016) 2461–2475.
- [130] A. Ardakani, C. Condo, W.J. Gross, Fast and efficient convolutional accelerator for edge computing, *IEEE Trans. Comput.* 69 (1) (2019) 138–152.
- [131] N.P. Jouppi, C. Young, N. Patil, D. Patterson, A domain-specific architecture for deep neural networks, *Commun. ACM* 61 (9) (2018) 50–59.
- [132] A. Moitra, A. Bhattacharjee, P. Panda, PIVOT-input-aware path selection for energy-efficient ViT inference, in: *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [133] Y. Zhang, L. Feng, H. Shan, Z. Zhu, A 109-gops/w fpga-based vision transformer accelerator with weight-loop dataflow featuring data reusing and resource saving, *IEEE Trans. Circuits Syst. Video Technol.* (2024).
- [134] K. Marino, P. Zhang, V.K. Prasanna, ME-ViT: A single-load memory-efficient FPGA accelerator for vision transformers, in: *2023 IEEE 30th International Conference on High Performance Computing, Data, and Analytics, HiPC, IEEE*, 2023, pp. 213–223.
- [135] N. Silberman, D. Hoiem, P. Kohli, R. Fergus, Indoor segmentation and support inference from rgb-d images, in: *Computer Vision–ECCV 2012: 12th European Conference on Computer Vision*, Florence, Italy, October 7–13, 2012, *Proceedings, Part V 12*, Springer, 2012, pp. 746–760.
- [136] H. Sharma, J. Park, D. Mahajan, E. Amaro, J.K. Kim, C. Shao, A. Mishra, H. Esmaeilzadeh, From high-level deep neural models to FPGAs, in: *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO*, 2016, pp. 1–12, <http://dx.doi.org/10.1109/MICRO.2016.7783720>.
- [137] T. Wang, K. Wang, H. Cai, J. Lin, Z. Liu, H. Wang, Y. Lin, S. Han, Apq: Joint search for network architecture, pruning and quantization policy, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2078–2087.
- [138] H. Cai, L. Zhu, S. Han, ProxylessNAS: Direct neural architecture search on target task and hardware, in: *International Conference on Learning Representations*, 2019, URL <https://arxiv.org/pdf/1812.00332.pdf>.
- [139] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration, *Proc. Mach. Learn. Syst.* 6 (2024) 87–100.
- [140] M.P. Véstias, R.P. Duarte, J.T. de Sousa, H.C. Neto, Fast convolutional neural networks in low density FPGAs using zero-skipping and weight pruning, *Electron.* 8 (11) (2019) 1321.
- [141] M. Liu, C. Zhou, S. Qiu, Y. He, H. Jiao, CNN accelerator at the edge with adaptive zero skipping and sparsity-driven data flow, *IEEE Trans. Circuits Syst. Video Technol.* 33 (12) (2023) 7084–7095, <http://dx.doi.org/10.1109/TCSVT.2023.3274964>.
- [142] G. Chen, Y. Guo, C. Li, F. Wang, D. Gong, L. Yuan, A subspace sparsity driven knowledge transfer strategy for dynamic constrained multiobjective optimization, *IEEE Trans. Evol. Comput.* (2025) <http://dx.doi.org/10.1109/TEVC.2025.3525635>, 1–1.
- [143] H. Pham Minh, N. Nguyen Xuan, S. Tran Thai, TT-vit: Vision transformer compression using tensor-train decomposition, in: *International Conference on Computational Collective Intelligence*, Springer, 2022, pp. 755–767.
- [144] J. Yan, K. Ando, J. Yu, M. Motomura, TT-MLP: Tensor train decomposition on deep MLPs, *IEEE Access* 11 (2023) 10398–10411, <http://dx.doi.org/10.1109/ACCESS.2023.3240784>.
- [145] Z. Qu, L. Deng, B. Wang, H. Chen, J. Lin, L. Liang, G. Li, Z. Zhang, Y. Xie, Hardware-enabled efficient data processing with tensor-train decomposition, *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.* 41 (2) (2021) 372–385.
- [146] X. Chen, Z. Liu, H. Tang, L. Yi, H. Zhao, S. Han, SparseViT: Revisiting activation sparsity for efficient high-resolution vision transformer, in: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 2061–2070.
- [147] Y. Dai, Y. Gao, F. Liu, Transmed: Transformers advance multi-modal medical image classification, *Diagnostics* 11 (8) (2021) 1384.
- [148] R. Raj, J. Mathew, S.K. Kannath, J. Rajan, StrokeViT with automl for brain stroke classification, *Eng. Appl. Artif. Intell.* 119 (2023) 105772.
- [149] B. Gheflati, H. Rivaz, Vision transformers for classification of breast ultrasound images, in: *2022 44th Annual International Conference of the IEEE Engineering in Medicine & Biology Society, EMBC, IEEE*, 2022, pp. 480–483.
- [150] A. Hatamizadeh, V. Nath, Y. Tang, D. Yang, H.R. Roth, D. Xu, Swin unetr: Swin transformers for semantic segmentation of brain tumors in mri images, in: *International MICCAI Brainlesion Workshop*, Springer, 2021, pp. 272–284.
- [151] M. Heidari, A. Kazerouni, M. Soltany, R. Azad, E.K. Aghdam, J. Cohen-Adad, D. Merhof, Hiformer: Hierarchical multi-scale representations using transformers for medical image segmentation, in: *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2023, pp. 6202–6212.
- [152] A. He, K. Wang, T. Li, C. Du, S. Xia, H. Fu, H2former: An efficient hierarchical hybrid transformer for medical image segmentation, *IEEE Trans. Med. Imaging* (2023).
- [153] J. Yang, L. Jiao, R. Shang, X. Liu, R. Li, L. Xu, EPT-net: Edge perception transformer for 3D medical image segmentation, *IEEE Trans. Med. Imaging* (2023).
- [154] Y. Shou, T. Meng, W. Ai, C. Xie, H. Liu, Y. Wang, Object detection in medical images based on hierarchical transformer and mask mechanism, *Comput. Intell. Neurosci.* 2022 (2022).
- [155] B. Leng, C. Wang, M. Leng, M. Ge, W. Dong, Deep learning detection network for peripheral blood leukocytes based on improved detection transformer, *Biomed. Signal Process. Control.* 82 (2023) 104518.
- [156] B. Wittmann, S. Shit, F. Navarro, J.C. Peeken, S.E. Combs, bjoern menze, SwinFPN: Leveraging vision transformers for 3D organs-at-risk detection, in: *Medical Imaging with Deep Learning*, 2022, URL <https://openreview.net/forum?id=yilz7DhgRU5>.
- [157] Z. Lin, Z. He, S. Xie, X. Wang, J. Tan, J. Lu, B. Tan, Aanet: Adaptive attention network for COVID-19 detection from chest X-ray images, *IEEE Trans. Neural Networks Learn. Syst.* 32 (11) (2021) 4781–4792.
- [158] L. Hou, Y. Cheng, N. Shazeer, N. Parmar, Y. Li, P. Korfiatis, T.M. Drucker, D.J. Blezek, X. Song, High resolution medical image analysis with spatial partitioning, 2019, arXiv preprint [arXiv:1909.03108](https://arxiv.org/abs/1909.03108).
- [159] L. Peng, H. Liang, G. Luo, T. Li, J. Sun, Rethinking transfer learning for medical image classification, in: *34th British Machine Vision Conference 2023, BMVC 2023, Aberdeen, UK, November 20–24, 2023, BMVA*, 2023, URL <https://papers.bmvc2023.org/0881.pdf>.
- [160] G. Vrbancić, V. Podgorelec, Transfer learning with adaptive fine-tuning, *IEEE Access* 8 (2020) 196197–196211.
- [161] M. Blott, T.B. Preußer, N.J. Fraser, G. Gambardella, K. O'Brien, Y. Umuroglu, M. Leeser, C. Vissers, FINN-r: An end-to-end deep-learning framework for fast exploration of quantized neural networks, *ACM Trans. Reconfigurable Technol. Syst. (TRET)* 11 (3) (2018) 1–23.



Shaibal Saha received his B.S. degree in Computer Science and Engineering from North South University, Bangladesh and M.S. degree in Computer Science from Oakland University in 2024. He is currently pursuing a Ph.D. degree in the Department of Computer Science and Engineering at Oakland University, USA, supervised by Dr. Lanyu Xu. His research interests include edge computing, deep learning and efficient model compression for edge devices.



Lanyu Xu received the B.S. degree in Computer Science from Tongji University, Shanghai, China, in 2015, and the Ph.D. degree in Computer Science from Wayne State University, Michigan, USA, in 2021. She is currently an Assistant Professor in the Department of Computer Science and Engineering at Oakland University, Michigan, where she directs the Edge Intelligence Systems Laboratory. Her research interests include edge computing, applied artificial intelligence, and connected health.